Text Analytics and Sentiment Analysis

(CT107-3-3-TXSA)

Group Assignment

Lecturer Name : Assoc. Prof. Dr. Nirase Fathima Abubacker

Hand out Date : 23 February 2026

Hand in Date : 8 May 2026

Intake Code : APU3F2601CS(DA) / APD3F2601CS(DA)

Group No : 3

Group Members

Student ID	Student Name
TP068983	Chan Chun Ming
TP068580	Eu Jun Hong
TP068495	Lim Beng Rhui (*Group Leader*)
TP067678	Ng Xuan Jack

Table of Contents

Introduction ····· 4

Part A: Preprocessing Techniques in NLP ····· 5

A.1 Tokenisation and Text Filtering ····· 5

A.1.1 Word Tokenisation Implementation ····· 5

A.1.2 Justification for Word Tokenisation Approach ····· 6

A.1.3 Stop Words and Punctuations Removal ····· 7

A.1.4 Importance of Stop Words and Punctuation Removal ····· 8

A.2 Word Stemming ····· 9

A.2.1 Importance of Word Stemming ····· 9

A.2.2 Implementation of Word Stemming in Python ····· 9

A.2.3 Comparative Analysis of Stemming Algorithms ····· 10

A.3 POS Tagging and Parse Trees Analysis ····· 12

A.3.1 POS Tagging Implementation ····· 12

A.3.2 Difference between POS Taggers ····· 13

A.3.3 Parse Trees ····· 14

A.4 Sentence Probability Analysis ····· 16

A.4.1 Sentence Probability using Unsmoothed Bigram Model ····· 16

A.4.2 Sentence Probability using Smoothed Bigram Model ····· 18

A.4.3 Python Implementation for Sentence Probability ····· 19

A.5 Alternative Tokenisation Approach ····· 20

A.5.1 Chan Chun Ming (TP068983): Moses Tokeniser ····· 20

A.5.2 Eu Jun Hong (TP068580): TextBlob Library ····· 22

A.5.3 Lim Beng Rhui (TP068495): Gensim Library ····· 23

A.5.4 Ng Xuan Jack (TP067678): spaCy Library ····· 25

Part B: Supervised Text Classification in NLP ····· 27

B.1 Dataset Selection and Exploratory Data Analysis ····· 27

B.1.1 Dataset Acquisition and Justification ····· 27

B.1.2 Exploratory Data Analysis ····· 28

B.1.3 Machine Learning Model Selection ····· 32

B.1.4 Data Preprocessing ····· 33

B.2 Supervised Text Classification ····· 37

B.3 Hyperparameter Selection and Optimisation ····· 42
B.3.1 Hyperparameters in Models ····· 42
B.3.2 Hyperparameter Optimisation Process ····· 43
B.4 Model Evaluation and Discussion ····· 46
B.4.1 Selection of Evaluation Metrics ····· 46
B.4.2 Performance and Results Presentation ····· 46
B.4.3 Critical Discussion on Model Performance ····· 47
Conclusion ····· 48
References ····· 49
Appendix A. Workload Matrix ····· 55
Appendix B. Ensemble Model ····· 56

Introduction

In recent years, the advancement of technology has led to the birth of sophisticated techniques, such as machine learning and artificial intelligence (AI), to assist human in performing their day-to-day tasks. However, as computers process information in binary formats, they lack the capability to learn human language semantically (Ali, 2018) and experience limitations when processing text and speech data. These challenges gave birth to the natural language processing (NLP) field, where attempts have been made to convert human text into machine-understandable representations, so that meaningful information can be derived from text to support data-driven decisions in domains like generative AI and voice assistants (Chen, 2024). Hence, this report aims to gain a better understanding towards the different processes involved in NLP by looking into different preprocessing and classification techniques, thereby identifying effective practices that can be applied in real-world applications.

Part A: Preprocessing Techniques in NLP

This section performs the different text preprocessing tasks involved in NLP using Python.

A.1 Tokenisation and Text Filtering

A.1.1 Word Tokenisation Implementation

Since the split function, regular expressions and the NLTK library are the common approaches implemented when performing word tokenisation (Ahmed, 2023), the Python code that implements these techniques are shown in Figures 1 to 3 together with its output.

Figure 1

Word Tokenisation using Split Function

```
# Retrieve the input text
input_text = data_1

# Create a list to store the tokenized words
split_tokens = []

# Document contains new lines, so loop through each line
for line in input_text.splitlines():

    # Split the line based on blank spaces
    split_line = line.split(" ")

    # Then only attach the split words to the list
    split_tokens.extend(split_line)

# Lastly, output the results
print(split_tokens)
```

✓ [2] 17ms

```
['Classification', 'is', 'the', 'task', 'of', 'choosing', 'the', 'correct', 'class', 'label', 'for', 'a', 'given', 'input.', 'In',
'basic', 'classification', 'tasks,', 'each', 'input', 'is', 'considered', 'in', 'isolation', 'from', 'all', 'other', 'inputs,', 'and',
'the', 'set', 'of', 'labels', 'is', 'defined', 'in', 'advance.', 'The', 'basic', 'classification', 'task', 'has', 'a', 'number', 'of',
'interesting', 'variants.', 'For', 'example,', 'in', 'multiclass', 'classification,', 'each', 'instance', 'may', 'be', 'assigned',
'multiple', 'labels;', 'in', 'open-class', 'classification,', 'the', 'set', 'of', 'labels', 'is', 'not', 'defined', 'in', 'advance;',
'and', 'in', 'sequence', 'classification,', 'a', 'list', 'of', 'inputs', 'are', 'jointly', 'classified.']
```

Figure 2

Word Tokenisation using Regular Expression

```
# Import library for regular expression
import re

# Retrieve the input text
input_text = data_1

# Perform splitting based on words
# \b means word boundary
# \w+ means one or more word character (letters, numbers, underscore)
regex_tokens = re.findall(r'\b\w+\b', input_text)
print(regex_tokens)
```

✓ [3] 30ms

```
['Classification', 'is', 'the', 'task', 'of', 'choosing', 'the', 'correct', 'class', 'label', 'for', 'a', 'given', 'input', 'In',
'basic', 'classification', 'tasks', 'each', 'input', 'is', 'considered', 'in', 'isolation', 'from', 'all', 'other', 'inputs', 'and',
'the', 'set', 'of', 'labels', 'is', 'defined', 'in', 'advance', 'The', 'basic', 'classification', 'task', 'has', 'a', 'number', 'of',
'interesting', 'variants', 'For', 'example', 'in', 'multiclass', 'classification', 'each', 'instance', 'may', 'be', 'assigned',
'multiple', 'labels', 'in', 'open', 'class', 'classification', 'the', 'set', 'of', 'labels', 'is', 'not', 'defined', 'in', 'advance',
'and', 'in', 'sequence', 'classification', 'a', 'list', 'of', 'inputs', 'are', 'jointly', 'classified']
```

Figure 3

Word Tokenisation using NLTK



```
# Import library for NLTK
import nltk

# Retrieve the input text
input_text = data_1

# Perform splitting using NLTK's word_tokenize function
nltk_tokens = nltk.tokenize.word_tokenize(input_text)
print(nltk_tokens)
```

✓ [4] 198ms

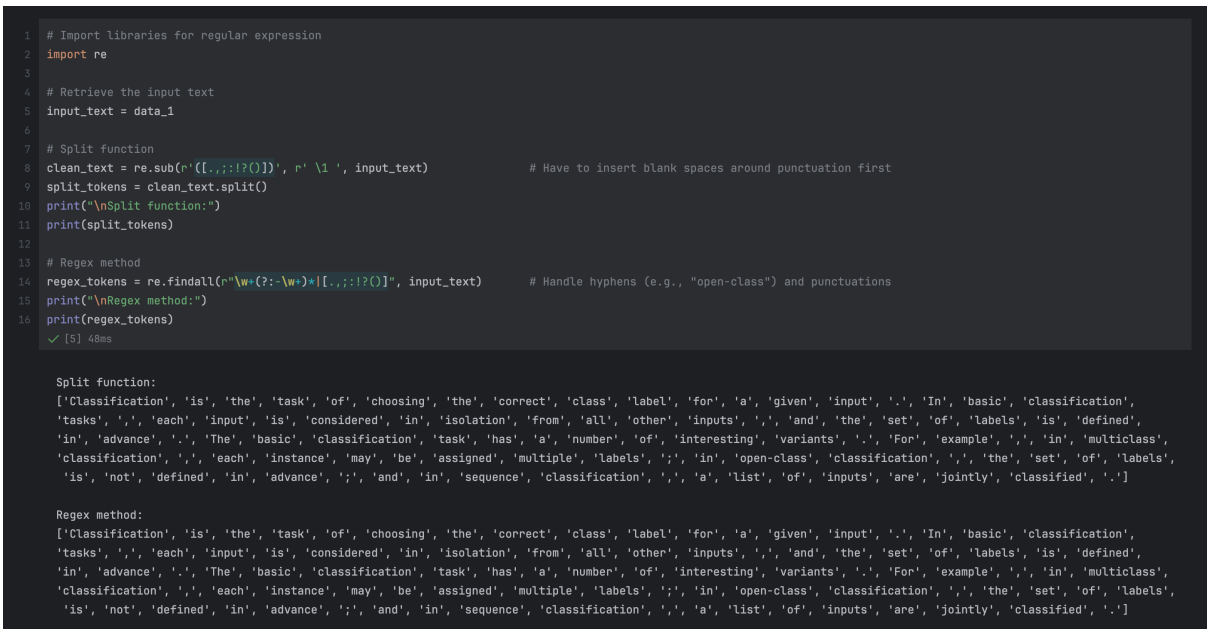
```
['Classification', 'is', 'the', 'task', 'of', 'choosing', 'the', 'correct', 'class', 'label', 'for', 'a', 'given', 'input', '.', 'In', 'basic', 'classification',
'tasks', ',', 'each', 'input', 'is', 'considered', 'in', 'isolation', 'from', 'all', 'other', 'inputs', ',', 'and', 'the', 'set', 'of', 'labels', 'is', 'defined',
'in', 'advance', '.', 'The', 'basic', 'classification', 'task', 'has', 'a', 'number', 'of', 'interesting', 'variants', '.', 'For', 'example', ',', 'in', 'multiclass',
'classification', ',', 'each', 'instance', 'may', 'be', 'assigned', 'multiple', 'labels', ';', 'in', 'open-class', 'classification', ',', 'the', 'set', 'of', 'labels',
 'is', 'not', 'defined', 'in', 'advance', ';', 'and', 'in', 'sequence', 'classification', ',', 'a', 'list', 'of', 'inputs', 'are', 'jointly', 'classified', '.']
```

A.1.2 Justification for Word Tokenisation Approach

Based on the outputs, the best approach to perform word tokenisation would be the `word_tokenize()` function from the NLTK library. As shown in Figure 3, the NLTK approach is able to separate punctuations from words while preserving the structure of original sentences, while to achieve similar outputs, additional preprocessing (see Figure 4) must be made towards the split function and regular expressions, especially when handling special words like `don't` and `U.S.A.`. Taking the token `input.` as an example, the split function treats it as a single token and give `input.`, regular expression ignores punctuations and yield `input`, while NLTK separates it into `input` and `.`. By having words and punctuations separated, this ensures uniformity among tokens (Jurafsky & Martin, 2026) to facilitate subsequent tasks.

Figure 4

Improvements for Tokenisation Using the Split Function and Regular Expression



```
# Import libraries for regular expression
import re

# Retrieve the input text
input_text = data_1

# Split function
clean_text = re.sub(r'([.,;:!?()])', r' \1 ', input_text)              # Have to insert blank spaces around punctuation first
split_tokens = clean_text.split()
print("\nSplit function:")
print(split_tokens)

# Regex method
regex_tokens = re.findall(r"\w+(?:-\w+)*|[.,;:!?()]", input_text)      # Handle hyphens (e.g., "open-class") and punctuations
print("\nRegex method:")
print(regex_tokens)
```

✓ [5] 48ms

```
Split function:
['Classification', 'is', 'the', 'task', 'of', 'choosing', 'the', 'correct', 'class', 'label', 'for', 'a', 'given', 'input', '.', 'In', 'basic', 'classification',
'tasks', ',', 'each', 'input', 'is', 'considered', 'in', 'isolation', 'from', 'all', 'other', 'inputs', ',', 'and', 'the', 'set', 'of', 'labels', 'is', 'defined',
'in', 'advance', '.', 'The', 'basic', 'classification', 'task', 'has', 'a', 'number', 'of', 'interesting', 'variants', '.', 'For', 'example', ',', 'in', 'multiclass',
'classification', ',', 'each', 'instance', 'may', 'be', 'assigned', 'multiple', 'labels', ';', 'in', 'open-class', 'classification', ',', 'the', 'set', 'of', 'labels',
 'is', 'not', 'defined', 'in', 'advance', ';', 'and', 'in', 'sequence', 'classification', ',', 'a', 'list', 'of', 'inputs', 'are', 'jointly', 'classified', '.']

Regex method:
['Classification', 'is', 'the', 'task', 'of', 'choosing', 'the', 'correct', 'class', 'label', 'for', 'a', 'given', 'input', '.', 'In', 'basic', 'classification',
'tasks', ',', 'each', 'input', 'is', 'considered', 'in', 'isolation', 'from', 'all', 'other', 'inputs', ',', 'and', 'the', 'set', 'of', 'labels', 'is', 'defined',
'in', 'advance', '.', 'The', 'basic', 'classification', 'task', 'has', 'a', 'number', 'of', 'interesting', 'variants', '.', 'For', 'example', ',', 'in', 'multiclass',
'classification', ',', 'each', 'instance', 'may', 'be', 'assigned', 'multiple', 'labels', ';', 'in', 'open-class', 'classification', ',', 'the', 'set', 'of', 'labels',
 'is', 'not', 'defined', 'in', 'advance', ';', 'and', 'in', 'sequence', 'classification', ',', 'a', 'list', 'of', 'inputs', 'are', 'jointly', 'classified', '.']
```

A.1.3 Stop Words and Punctuations Removal

Since the NLTK library contains a built-in corpus with stop words and gives the best result during tokenisation, the library is used to perform text filtering as shown in Figure 5.

Figure 5

Stop Words and Punctuations Removal using NLTK

```python
# Library import
from nltk.corpus import stopwords
import nltk
import string           # string library is used to handle punctuations

# Download the stop word corpus and extract the English stop words
nltk.download('stopwords')
stop_words = set(stopwords.words('english'))

# Retrieve the NLTK tokens from previous step
tokens = nltk_tokens

# Separate the tokens into clean tokens, stop words and punctuations
clean_tokens = []
found_stop_words = []
found_punctuations = []

# Loop through each token to categorize them
for token in tokens:

    token = token.lower()       # Convert to lowercase so that tokens can match the stop words from corpus

    if token in stop_words:
        found_stop_words.append(token)

    elif token in string.punctuation:
        found_punctuations.append(token)

    else:
        clean_tokens.append(token)

# Output the results
print("Clean Tokens:")
print(clean_tokens)

print("\nStop Words Found:")
print(set(found_stop_words))

print("\nPunctuations Found:")
print(set(found_punctuations))
```

✓ [8] 66ms

```
Clean Tokens:
['classification', 'task', 'choosing', 'correct', 'class', 'label', 'given', 'input', 'basic', 'classification', 'tasks', 'input',
 'considered', 'isolation', 'inputs', 'set', 'labels', 'defined', 'advance', 'basic', 'classification', 'task', 'number', 'interesting',
 'variants', 'example', 'multiclass', 'classification', 'instance', 'may', 'assigned', 'multiple', 'labels', 'open-class',
 'classification', 'set', 'labels', 'defined', 'advance', 'sequence', 'classification', 'list', 'inputs', 'jointly', 'classified']

Stop Words Found:
{'be', 'is', 'not', 'are', 'from', 'for', 'and', 'the', 'in', 'all', 'other', 'of', 'a', 'each', 'has'}

Punctuations Found:
{',', ';', '.'}
[nltk_data] Downloading package stopwords to
[nltk_data]     /Users/limbengrhui/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

A.1.4 Importance of Stop Words and Punctuation Removal

When performing text analytics tasks, stop words and punctuations are often removed due to the following reasons:

- **To improve the quality of textual data**

  Stop words, such as `be`, `is`, and `of`, as well as punctuations like commas (`,`) and full stops (`.`), often carry little semantic value and does not contribute towards the conceptual meaning of sentences. By removing these elements, noise in the dataset can be reduced, so that models can focus on keywords that offer more meaningful insights to improve its quality and accuracy (Sarica & Luo, 2021).

- **To reduce text dimensionality**

  Stop words and punctuations appear frequently in documents, which increases the number of tokens and features generated when vectorisation methods like Bag-of-Words are performed. According to Kaur and Buttar (2018), this drastically increases the computational cost and memory required but not contributing to models' prediction performance. Hence, these elements shall be removed to improve training efficiency.

A.2 Word Stemming

A.2.1 Importance of Word Stemming

Stemming refers to a technique in text preprocessing that reduce the variant forms of words and standardises them into their respective root or base form (Flores & Moreira, 2016). Some of the importance of performing word stemming include:

- **Reduce dimensionality**

  By converting words into their root forms, stemming helps to reduce dimensionality of text data during vectorisation operations. For instance, words like `classification` and `classified`, despite having the same base form called `classify`, will not be treated as a single entity if stemming is not performed, leading to redundant word representation.

- **Enhance model performance and efficiency**

  By performing stemming to group similar words and reduce the vocabulary size of the dataset, classification models will be able to generalize textual patterns more effectively, not only reducing the memory space required for subsequent preprocessing tasks but also improving the efficiency and accuracy of classification models (Aljumily, 2024).

A.2.2 Implementation of Word Stemming in Python

Three approaches are implemented to apply word stemming, including using regular expressions, the `PorterStemmer` algorithm and the `LancasterStemmer` algorithm, with their respective code snippets and outputs shown in Figures 6 to 8.

Figure 6

Word Stemming using Regular Expression



```
# A function to remove suffix using regex
def regex_stem(word):
    return re.sub(r"(ing|ed|ly|es|s|ion)$", "", word)

# Apply stemming using regex for each cleaned token (from Q1)
regex_stemmed_tokens = [regex_stem(token) for token in clean_tokens]

# Output and compare the results
print("Original Clean Tokens:")
print(clean_tokens)
print("\nRegex Stemmed Tokens:")
print(regex_stemmed_tokens)
✓ [7] 42ms

Original Clean Tokens:
['classification', 'task', 'choosing', 'correct', 'class', 'label', 'given', 'input', 'basic', 'classification', 'tasks', 'input', 'considered', 'isolation', 'inputs',
'set', 'labels', 'defined', 'advance', 'basic', 'classification', 'task', 'number', 'interesting', 'variants', 'example', 'multiclass', 'classification', 'instance',
'may', 'assigned', 'multiple', 'labels', 'open-class', 'classification', 'set', 'labels', 'defined', 'advance', 'sequence', 'classification', 'list', 'inputs',
'jointly', 'classified']

Regex Stemmed Tokens:
['classificat', 'task', 'choos', 'correct', 'clas', 'label', 'given', 'input', 'basic', 'classificat', 'task', 'input', 'consider', 'isolat', 'input', 'set', 'label',
'defin', 'advance', 'basic', 'classificat', 'task', 'number', 'interest', 'variant', 'example', 'multiclas', 'classificat', 'instance', 'may', 'assign', 'multiple',
'label', 'open-clas', 'classificat', 'set', 'label', 'defin', 'advance', 'sequence', 'classificat', 'list', 'input', 'joint', 'classifi']
```

Figure 7

Word Stemming using `PorterStemmer`

```
# Import library for PorterStemmer
from nltk.stem import PorterStemmer

# Create an instance of PorterStemmer
porter_stemmer = PorterStemmer()

# Perform stemming on each cleaned token using PorterStemmer
porter_stemmed_tokens = [porter_stemmer.stem(token) for token in clean_tokens]

# Output and compare the results
print("Original Clean Tokens:")
print(clean_tokens)
print("\nPorter Stemmed Tokens:")
print(porter_stemmed_tokens)
```

✓ [8] 64ms

```
Original Clean Tokens:
['classification', 'task', 'choosing', 'correct', 'class', 'label', 'given', 'input', 'basic', 'classification', 'tasks', 'input', 'considered', 'isolation', 'inputs',
'set', 'labels', 'defined', 'advance', 'basic', 'classification', 'task', 'number', 'interesting', 'variants', 'example', 'multiclass', 'classification', 'instance',
'may', 'assigned', 'multiple', 'labels', 'open-class', 'classification', 'set', 'labels', 'defined', 'advance', 'sequence', 'classification', 'list', 'inputs',
'jointly', 'classified']

Porter Stemmed Tokens:
['classif', 'task', 'choos', 'correct', 'class', 'label', 'given', 'input', 'basic', 'classif', 'task', 'input', 'consid', 'isol', 'input', 'set', 'label', 'defin',
'advanc', 'basic', 'classif', 'task', 'number', 'interest', 'variant', 'exampl', 'multiclass', 'classif', 'instanc', 'may', 'assign', 'multipl', 'label', 'open-class',
'classif', 'set', 'label', 'defin', 'advanc', 'sequenc', 'classif', 'list', 'input', 'jointli', 'classifi']
```

Figure 8

Word Stemming using `LancasterStemmer`

```
# Import library for LancasterStemmer
from nltk.stem import LancasterStemmer

# Create an instance of LancasterStemmer
lancaster_stemmer = LancasterStemmer()

# Perform stemming on each cleaned token using LancasterStemmer
lancaster_stemmed_tokens = [lancaster_stemmer.stem(token) for token in clean_tokens]

# Output and compare the results
print("Original Clean Tokens:")
print(clean_tokens)
print("\nLancaster Stemmed Tokens:")
print(lancaster_stemmed_tokens)
```

✓ [9] 66ms

```
Original Clean Tokens:
['classification', 'task', 'choosing', 'correct', 'class', 'label', 'given', 'input', 'basic', 'classification', 'tasks', 'input', 'considered', 'isolation', 'inputs',
'set', 'labels', 'defined', 'advance', 'basic', 'classification', 'task', 'number', 'interesting', 'variants', 'example', 'multiclass', 'classification', 'instance',
'may', 'assigned', 'multiple', 'labels', 'open-class', 'classification', 'set', 'labels', 'defined', 'advance', 'sequence', 'classification', 'list', 'inputs',
'jointly', 'classified']

Lancaster Stemmed Tokens:
['class', 'task', 'choos', 'correct', 'class', 'label', 'giv', 'input', 'bas', 'class', 'task', 'input', 'consid', 'isol', 'input', 'set', 'label', 'defin', 'adv',
'bas', 'class', 'task', 'numb', 'interest', 'vary', 'exampl', 'multiclass', 'class', 'inst', 'may', 'assign', 'multipl', 'label', 'open-class', 'class', 'set', 'label',
'defin', 'adv', 'sequ', 'class', 'list', 'input', 'joint', 'class']
```

A.2.3 Comparative Analysis of Stemming Algorithms

Based on the outputs, the difference between each stemming approach is evaluated and outlined in Table 1. In overall, the `PorterStemmer` stands out to be the best approach as it provides a strong balance between speed, efficiency and accuracy when preprocessing texts in English. As `PorterStemmer` is able to maintain the semantic meaning of words while effectively reducing the vocabulary size of the dataset, this makes it suitable to be applied to text classification tasks that involves English texts.

Table 1

Comparison between Different Stemming Approaches

Aspect	Regular Expression Stemmer	`PorterStemmer`	`LancasterStemmer`
Definition	A flexible stemming algorithm that allows users to define custom rules with regex (regular expression)	One of the most widely used stemming algorithms which applied rule-based to remove common suffixes from English words.	Well-known aggressive rule based stemming algorithm. The original word will be processed in iterative manner to remove suffix on defined rules.
Observed outputs	• `classification` → `classificat` • `class` → `clas` • `basic` → `basic`	• `classification` → `classif` • `class` → `class` • `Basic` → `basic`	• `classification` → `class` • `class` → `class` • `basic` → `bas`
Limitation	• Requires manual rule definition • May not be able to cover all scenario without proper consideration	• Restricted to English only	• Over-stemming due to its aggressive manner • May lead to word distortion and reduce interpretability
Advantages	• Highly customizable • High suitability for domain specific tasks	• Fast and efficient due to its simplicity • Results are balanced and meaningful	• Fast preprocessing in small datasets

Note. Compiled and adapted from GeeksforGeeks (2025), Jivani (2011), and Murel and Kavlakoglu (2023).

A.3 POS Tagging and Parse Trees Analysis

A.3.1 POS Tagging Implementation

Several POS tagging methods are implemented using different libraries including regular expression, NLTK, and TextBlob, with the outputs shown in Figures 9 to 11.

Figure 9

POS Tagging using Regular Expression

```
# NLTK for tokenization
import nltk
from nltk.tokenize import word_tokenize

# Retrieve data
text_data = data_2

# First, perform tokenization to divide sentences into words
tokens = word_tokenize(text_data)

# Define the RE patterns to be used
patterns = [

    (r'^\d+(\.\d+)?$', 'CD'),                          # Cardinal number (integer or decimal)

    (r'.*ed$', 'VBD'),                                 # Past tense verbs (e.g., walked, barked)
    (r'.*ing$', 'VBG'),                                # Gerunds or present participles (e.g., walking, barking)
    (r'.*es$', 'VBZ'),                                 # 3rd person singular present verbs (e.g., watches, goes)

    (r'.*ly$', 'RB'),                                  # Adverbs ending with -ly (e.g., quickly, silently)
    (r'.*ful$', 'JJ'),                                 # Adjectives ending with -ful (e.g., beautiful, helpful)
    (r'.*able$', 'JJ'),                                # Adjectives ending with -able (e.g., readable, comfortable)
    (r'.*ous$', 'JJ'),                                 # Adjectives ending with -ous (e.g., dangerous, curious)

    (r'.*big$', 'JJ'),                                 # Specific adjective: big
    (r'.*black$', 'JJ'),                               # Specific adjective: black
    (r'.*white$', 'JJ'),                               # Specific adjective: white

    (r'^(the|a|an)$', 'DT'),                           # Determiners / articles (e.g., the, a, an)
    (r'^(and|or|but)$', 'CC'),                         # Coordinating conjunctions (e.g., and, or, but)
    (r'^(at|in|on|by|with|from|to|of|for)$', 'IN'),    # Prepositions (e.g., at, in, on, by, with, from, to, of, for)

    (r'\.$', '.'),                                     # Full stop punctuation
    (r'.*', 'NN')                                      # Default to noun (NN) for any word that doesn't match the above patterns
]

# Use RegexpTagger to tag each word with the defined pattern and output results
regex_tagger = nltk.RegexpTagger(patterns)
regex_tagging = regex_tagger.tag(tokens)
print(regex_tagging)
✓ [9] 64ms

[('The', 'NN'), ('big', 'JJ'), ('black', 'JJ'), ('dog', 'NN'), ('barked', 'VBD'), ('at', 'IN'), ('the', 'DT'), ('white', 'JJ'), ('cat', 'NN'), ('and', 'CC'),
('chased', 'VBD'), ('away', 'NN'), ('.', '.')]
```

Figure 10

POS Tagging using NLTK

```
# Library import
import nltk
from nltk.tokenize import word_tokenize

# Retrieve data
text_data = data_2

# Perform tokenization using NLTK functions
tokens = word_tokenize(text_data)

# Perform POS tagging using NLTK
nltk_tagging = nltk.pos_tag(tokens)
print(nltk_tagging)
✓ [7] 83ms

[('The', 'DT'), ('big', 'JJ'), ('black', 'JJ'), ('dog', 'NN'), ('barked', 'VBD'), ('at', 'IN'), ('the', 'DT'), ('white', 'JJ'), ('cat', 'NN'), ('and', 'CC'),
('chased', 'VBD'), ('away', 'RB'), ('.', '.')]
```

Figure 11

POS Tagging using TextBlob

```
# Library import
from textblob import TextBlob

# Retrieve dataset
text_data = data_2

# Perform POS tagging using TextBlob
textblob = TextBlob(text_data)
print(textblob.tags)
✓ [8] 33ms

[('The', 'DT'), ('big', 'JJ'), ('black', 'JJ'), ('dog', 'NN'), ('barked', 'VBD'), ('at', 'IN'), ('the', 'DT'), ('white', 'JJ'), ('cat', 'NN'), ('and', 'CC'),
('chased', 'VBD'), ('away', 'RB')]
```

A.3.2 Difference between POS Taggers

The difference between each POS Tagger is evaluated based on the following aspects:

- **NLTK POS Tagger and TextBlob POS Tagger**
  When performing POS tagging using the NLTK and TextBlob libraries, both approaches are able to provide identical tags for the sentence in `Data_2.txt`, where words like `big`, `dog` and `barked` are accurately identified as adjective (JJ), noun (NN) and past-tense verb (VBD) respectively. The main difference between NLTK and TextBlob is how punctuations are handled, where the NLTK tagger assigns punctuations like commas (`,`) or full stops (`.`) with their respective tags, while TextBlob removes punctuation during its preprocessing to perform POS tagging only on actual words.

- **Limitation of POS Tagging using Regular Expression**
  Instead of using trained models like NLTK and TextBlob, regular expressions rely on predefined rules to perform POS tagging, where words are evaluated solely based on their text patterns rather than grammatical context. Looking at Figure 9, words with special patterns, such as barked and chased that end with `-ed`, can be easily identified. However, when encountering adjectives like `big` and prepositions like `at`, manual tagging is required since these words do not match any predefined patterns, thus aligning with Pham (2020) who demonstrated the limitations of using rule-based methods for POS tagging when handling unseen or ambiguous words.

Overall, NLTK and TextBlob are more preferred to perform POS tagging as both approaches not only uses a simpler syntax, but are also able to accurately assign words with their corresponding tags. In contrast, the regular expression approach requires user to manually define rules, making it unable to capture contextual meaning and reduces its tagging accuracy.

A.3.3 Parse Trees

Before constructing the parse trees, the corresponding context-free grammar (CFG) must be defined first. Based on the sentence `The big black dog barked at the white cat and chased away.`, with reference to the NLTK documentation (Bird et al., 2009), the derived CFGs are:

Table 2

Context-Free Grammar Derived from `Data_2.txt`

Phrase Type	Phrase	Interpretation	Corresponding CFG
Noun Phrase	The big black dog	Article + Adjective + Adjective + Noun	NP -> DT JJ JJ NN
	The white cat	Article + Adjective + Noun	NP -> DT JJ NN
Verb Phrase	barked at the white cat	Verb + Prepositional phrase	VP -> VBD PP
	chased away	Verb + Adverb	VP -> VBD RB
	barked at the white cat + and + chased away	Verb phrase + Conjunction + Verb phrase	VP -> VP CC VP
Prepositional Phrase	at the white cat	Preposition + Noun phrase	PP -> IN NP
Top-Level Sentence Rule	The big black dog + barked at the white cat and chased away	Noun phrase + Verb phrase	S -> NP VP

Based on Table 2, the parse tree is constructed using the NLTK library, as shown in Figure 12.

Figure 12

Parse Tree Constructed using the NLTK Library

```
# Import NLTK library for prase tree generation
import nltk

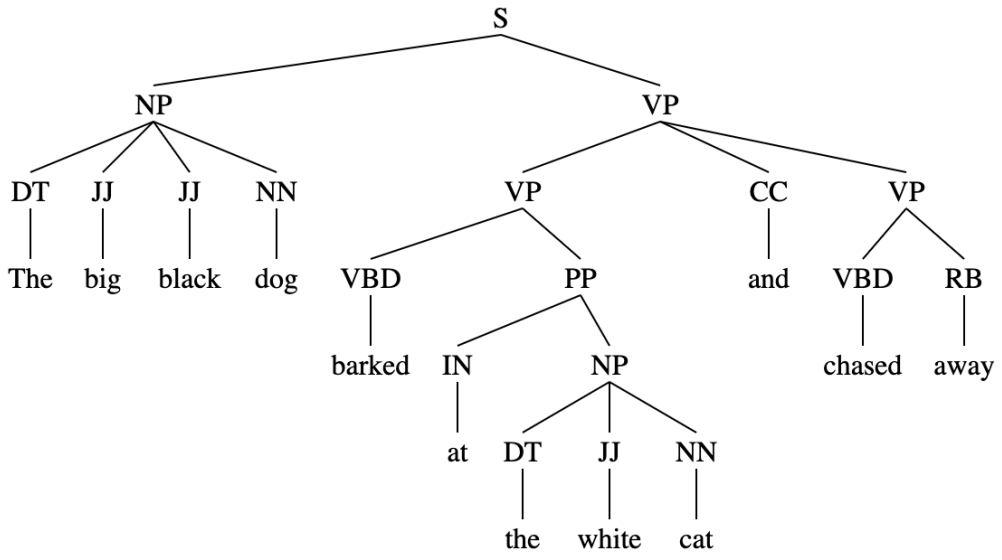
# Define the CFG
cfg = nltk.CFG.fromstring("""
S -> NP VP
NP -> DT JJ JJ NN | DT JJ NN
VP -> VBD PP | VBD RB | VP CC VP
PP -> IN NP

DT -> 'The' | 'the'
JJ -> 'big' | 'black' | 'white'
NN -> 'dog' | 'cat'
VBD -> 'barked' | 'chased'
IN -> 'at'
RB -> 'away'
CC -> 'and'
""")

# Retrieve data and eliminate full stop
sentence = data_2.replace('.', '')

# Create the parser from NLTK
parser = nltk.ChartParser(cfg)
for tree in parser.parse(nltk.word_tokenize(sentence)):
    display(tree)
```

✓ [20] 106ms



As the provided sentence does not exhibit any observable structural ambiguity that leads to multiple valid interpretations towards its contextual meaning, this aligns with the output in Figure 12 that displays only one parse tree.

A.4 Sentence Probability Analysis

A.4.1 Sentence Probability using Unsmoothed Bigram Model

To calculate sentence probabilities, bigram counts are first constructed based on the sentences provided in `Data_3.txt`, as shown in Figure 13.

Figure 13

Bigram Frequency Counts for Padded Sentences

Training Corpus

1. <s> He read a book </s>
2. <s> I read a different book </s>
3. <s> He read a book by Danielle </s>

Unigram Frequency Count

Word	Count
<s>	3
</s>	3
he	2
read	3
a	3
book	3
I	1
different	1
by	1
Danielle	1

Vocabulary size (V) = 10

Bigram Frequency Count

Bigram Pair	Count
<s> → He	2
<s> → I	1
He → read	2
I → read	1
read → a	3
a → book	2
a → different	1
different → book	1
book → </s>	2
book → by	1
by → Danielle	1
Danielle → </s>	1

To calculate the probability for the sentence `"<s> I read a different book by Danielle </s>"` using the unsmoothed bigram model, the following formula is used:

$$P(S) = \prod_{i=0}^{|S|} P(S_i \mid S_{i-1})$$

$$= P(\texttt{I} \mid \texttt{<s>}) \cdot P(\texttt{read} \mid \texttt{I}) \cdot P(\texttt{a} \mid \texttt{read}) \cdot P(\texttt{different} \mid \texttt{a}) \cdot P(\texttt{book} \mid \texttt{different})$$

$$\cdot P(\texttt{by} \mid \texttt{book}) \cdot P(\texttt{Danielle} \mid \texttt{by}) \cdot P(\texttt{</s>} \mid \texttt{Danielle})$$

The individual bigram probabilities are first calculated:

- $P(\texttt{I} \mid \texttt{<s>}) = \frac{C(\texttt{<s> , I})}{C(\texttt{<s>})} = \frac{1}{3}$
- $P(\texttt{read} \mid \texttt{I}) = \frac{C(\texttt{I , read})}{C(\texttt{I})} = \frac{1}{1} = 1$
- $P(\texttt{a} \mid \texttt{read}) = \frac{C(\texttt{read , a})}{C(\texttt{read})} = \frac{3}{3} = 1$
- $P(\texttt{different} \mid \texttt{a}) = \frac{C(\texttt{a , different})}{C(\texttt{a})} = \frac{1}{3}$
- $P(\texttt{book} \mid \texttt{different}) = \frac{C(\texttt{different , book})}{C(\texttt{different})} = \frac{1}{1} = 1$
- $P(\texttt{by} \mid \texttt{book}) = \frac{C(\texttt{book , by})}{C(\texttt{book})} = \frac{1}{3}$
- $P(\texttt{Danielle} \mid \texttt{by}) = \frac{C(\texttt{by , Danielle})}{C(\texttt{by})} = \frac{1}{1} = 1$
- $P(\texttt{</s>} \mid \texttt{Danielle}) = \frac{C(\texttt{Danielle , </s>})}{C(\texttt{Danielle})} = \frac{1}{1} = 1$

Then, the sentence probability based on the unsmoothed bigram model can be calculated:

$$P(S) = \frac{1}{3} \cdot 1 \cdot 1 \cdot \frac{1}{3} \cdot 1 \cdot \frac{1}{3} \cdot 1 \cdot 1 = \frac{1}{27} \approx 0.0370$$

A.4.2 Sentence Probability using Smoothed Bigram Model

To handle unseen bigrams, Laplace smoothing is applied by adding pseudo-counts so that the probability of unseen word combinations will not be zero (Hankare, 2024). By using the Laplace smoothing formula:

$$P(S_i \mid S_{i-1}) = \frac{C(S_{i-1}, S_i) + 1}{C(S_{i-1}) + V}$$

the probability of each individual bigram pair is calculated:

- $P(\texttt{I} \mid \texttt{<s>}) = \frac{C(\texttt{<s> , I}) + 1}{C(\texttt{<s>}) + 10} = \frac{2}{13}$
- $P(\texttt{read} \mid \texttt{I}) = \frac{C(\texttt{I , read}) + 1}{C(\texttt{I}) + 10} = \frac{2}{11}$
- $P(\texttt{a} \mid \texttt{read}) = \frac{C(\texttt{read , a}) + 1}{C(\texttt{read}) + 10} = \frac{4}{13}$
- $P(\texttt{different} \mid \texttt{a}) = \frac{C(\texttt{a , different}) + 1}{C(\texttt{a}) + 10} = \frac{2}{13}$
- $P(\texttt{book} \mid \texttt{different}) = \frac{C(\texttt{different , book}) + 1}{C(\texttt{different}) + 10} = \frac{2}{11}$
- $P(\texttt{by} \mid \texttt{book}) = \frac{C(\texttt{book , by}) + 1}{C(\texttt{book}) + 10} = \frac{2}{13}$
- $P(\texttt{Danielle} \mid \texttt{by}) = \frac{C(\texttt{by , Danielle}) + 1}{C(\texttt{by}) + 10} = \frac{2}{11}$
- $P(\texttt{</s>} \mid \texttt{Danielle}) = \frac{C(\texttt{Danielle , </s>}) + 1}{C(\texttt{Danielle}) + 10} = \frac{2}{11}$

Then, the sentence probability based on the smoothed bigram model is:

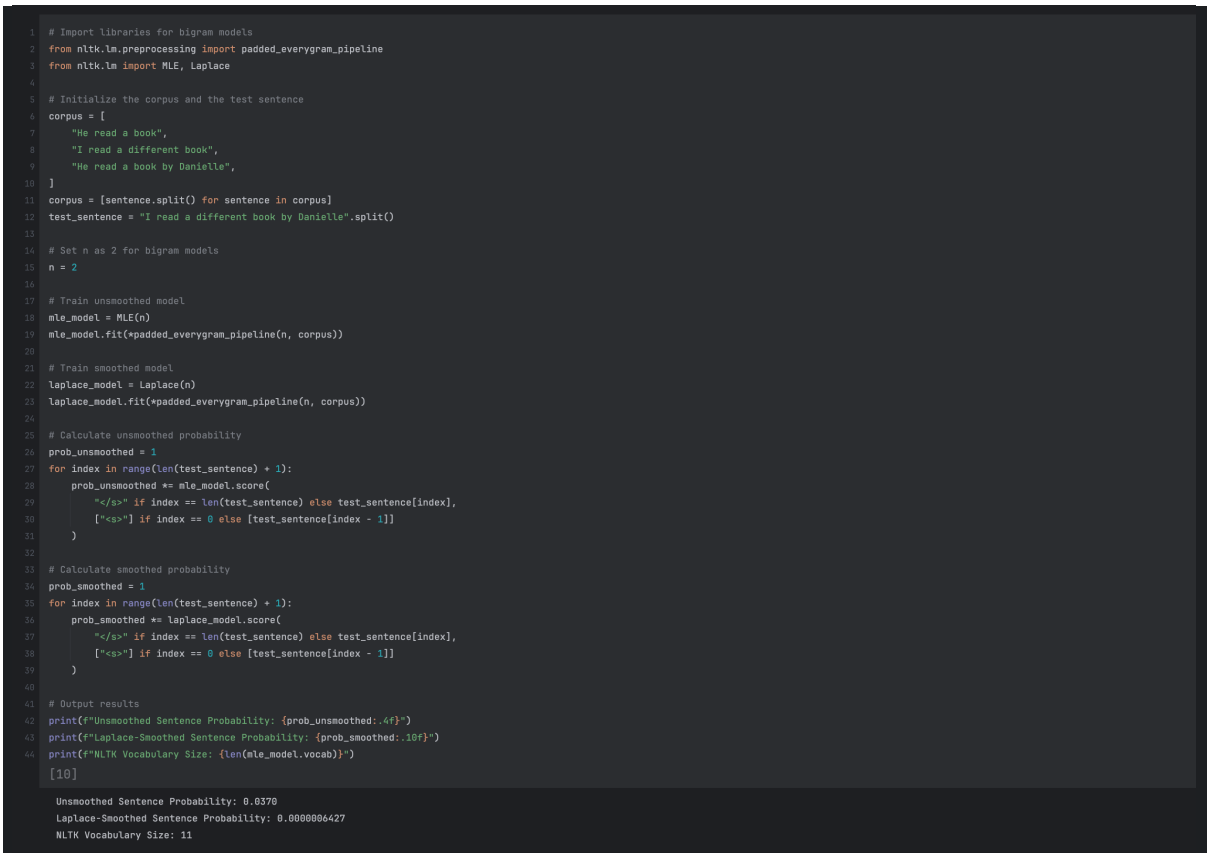
$$P(S) = \frac{2}{13} \cdot \frac{2}{11} \cdot \frac{4}{13} \cdot \frac{2}{13} \cdot \frac{2}{11} \cdot \frac{2}{13} \cdot \frac{2}{11} \cdot \frac{2}{11} = \frac{512}{418{,}161{,}601} \approx 1.224 \times 10^{-6}$$

A.4.3 Python Implementation for Sentence Probability

To verify the accuracy of the calculated values, the bigram model is implemented using the NLTK library in Python to calculate the sentence probabilities using unsmoothed and Laplace-smoothed models, as shown in Figure 14.

Figure 14

Python Implementation for Bigram Model to Calculate Sentence Probability



```python
# Import libraries for bigram models
from nltk.lm.preprocessing import padded_everygram_pipeline
from nltk.lm import MLE, Laplace

# Initialize the corpus and the test sentence
corpus = [
    "He read a book",
    "I read a different book",
    "He read a book by Danielle",
]
corpus = [sentence.split() for sentence in corpus]
test_sentence = "I read a different book by Danielle".split()

# Set n as 2 for bigram models
n = 2

# Train unsmoothed model
mle_model = MLE(n)
mle_model.fit(*padded_everygram_pipeline(n, corpus))

# Train smoothed model
laplace_model = Laplace(n)
laplace_model.fit(*padded_everygram_pipeline(n, corpus))

# Calculate unsmoothed probability
prob_unsmoothed = 1
for index in range(len(test_sentence) + 1):
    prob_unsmoothed *= mle_model.score(
        "</s>" if index == len(test_sentence) else test_sentence[index],
        ["<s>"] if index == 0 else [test_sentence[index - 1]]
    )

# Calculate smoothed probability
prob_smoothed = 1
for index in range(len(test_sentence) + 1):
    prob_smoothed *= laplace_model.score(
        "</s>" if index == len(test_sentence) else test_sentence[index],
        ["<s>"] if index == 0 else [test_sentence[index - 1]]
    )

# Output results
print(f"Unsmoothed Sentence Probability: {prob_unsmoothed:.4f}")
print(f"Laplace-Smoothed Sentence Probability: {prob_smoothed:.10f}")
print(f"NLTK Vocabulary Size: {len(mle_model.vocab)}")
[10]
```

```
Unsmoothed Sentence Probability: 0.0370
Laplace-Smoothed Sentence Probability: 0.0000006427
NLTK Vocabulary Size: 11
```

Based on the output, it is observed that a similar result is obtained for the unsmoothed sentence probability, which validates the manually calculated probability. However, a difference is observed in the Laplace-smoothed result, as NLTK includes an additional unseen vocabulary token called `<UNK>` during its computation (Bird et al., 2009), which increases the vocabulary size by 1 and gives a smaller probability compared to the result calculated manually. Hence, if the calculation of the NLTK library is replicated, the smoothed probability will be:

$$P(S) = \frac{2}{14} \cdot \frac{2}{12} \cdot \frac{4}{14} \cdot \frac{2}{14} \cdot \frac{2}{12} \cdot \frac{2}{14} \cdot \frac{2}{12} \cdot \frac{2}{12} = \frac{512}{796{,}594{,}176} \approx 6.4274 \times 10^{-7}$$

The recalculated result tallies with the output shown in Figure 14, thereby implying that the difference is caused by the way NLTK handles vocabulary size rather than calculation errors.

A.5 Alternative Tokenisation Approach

A.5.1 Chan Chun Ming (TP068983): Moses Tokeniser

Word Tokenisation Implementation

Developed for the Moses machine translation system, the Moses tokeniser is a rule-based tokenizer that splits text into tokens while handling punctuation and special cases based on linguistic rules (PythonPI, 2023). The Moses tokeniser is implemented in Figure 15 to compare with the `word_tokenization()` function approach from the NLTK library.

Figure 15

Word Tokenisation using Moses Tokeniser

```
# --- Remember to run pip install sacremoses ---
import sys
import subprocess
import importlib

# Code to automatically install sacremoses library if it's not available in the current env
try:
    sacremoses = importlib.import_module('sacremoses')
except ModuleNotFoundError:
    subprocess.check_call([sys.executable, "-m", "pip", "install", "sacremoses"])
    sacremoses = importlib.import_module('sacremoses')

# Access tokenizer classes from the imported module to avoid static import errors
MosesTokenizer = sacremoses.MosesTokenizer
MosesDetokenizer = sacremoses.MosesDetokenizer

# Initialize tokenizer / detokenizer
mt = MosesTokenizer()
mdet = MosesDetokenizer()

# Tokenize the original document `data_1`
text = data_1
moses_tokens = mt.tokenize(text, return_str=False)

# Display outputs
print("Moses tokenized (first 100 tokens):")
print(moses_tokens[:100])

print("\nNLTK tokens")
print(nltk_tokens)

print("\nDetokenized preview (first 500 chars):")
detok = mdet.detokenize(moses_tokens)
print(detok[:500])

print("\nDocument / token counts:")
print("Original characters:", len(text), ", Tokens:", len(moses_tokens))
✓ [15] 196ms

Moses tokenized (first 100 tokens):
['Classification', 'is', 'the', 'task', 'of', 'choosing', 'the', 'correct', 'class', 'label', 'for', 'a', 'given', 'input', '.', 'In', 'basic', 'classification',
 'tasks', ',', 'each', 'input', 'is', 'considered', 'in', 'isolation', 'from', 'all', 'other', 'inputs', ',', 'and', 'the', 'set', 'of', 'labels', 'is',
'defined', 'in', 'advance', '.', 'The', 'basic', 'classification', 'task', 'has', 'a', 'number', 'of', 'interesting', 'variants', '.', 'For', 'example', ',',
'in', 'multiclass', 'classification', ',', 'each', 'instance', 'may', 'be', 'assigned', 'multiple', 'labels', ';', 'in', 'open-class', 'classification', ',',
'the', 'set', 'of', 'labels', 'is', 'not', 'defined', 'in', 'advance', ';', 'and', 'in', 'sequence', 'classification', ',', 'a', 'list', 'of', 'inputs', 'are',
'jointly', 'classified', '.']

NLTK tokens
['Classification', 'is', 'the', 'task', 'of', 'choosing', 'the', 'correct', 'class', 'label', 'for', 'a', 'given', 'input', '.', 'In', 'basic', 'classification',
 'tasks', ',', 'each', 'input', 'is', 'considered', 'in', 'isolation', 'from', 'all', 'other', 'inputs', ',', 'and', 'the', 'set', 'of', 'labels', 'is',
'defined', 'in', 'advance', '.', 'The', 'basic', 'classification', 'task', 'has', 'a', 'number', 'of', 'interesting', 'variants', '.', 'For', 'example', ',',
'in', 'multiclass', 'classification', ',', 'each', 'instance', 'may', 'be', 'assigned', 'multiple', 'labels', ';', 'in', 'open-class', 'classification', ',',
'the', 'set', 'of', 'labels', 'is', 'not', 'defined', 'in', 'advance', ';', 'and', 'in', 'sequence', 'classification', ',', 'a', 'list', 'of', 'inputs', 'are',
'jointly', 'classified', '.']

Detokenized preview (first 500 chars):
Classification is the task of choosing the correct class label for a given input. In basic classification tasks, each input is considered in isolation from all
other inputs, and the set of labels is defined in advance. The basic classification task has a number of interesting variants. For example, in multiclass
classification, each instance may be assigned multiple labels; in open-class classification, the set of labels is not defined in advance; and in sequence
classification, a list of inputs

Document / token counts:
Original characters: 524 , Tokens: 94
```

Difference between Moses and NLT

From the results shown in Figure 15, both NLTK and `MosesTokenizer` approach produce the same tokens for the text data, including handling of punctuation and special cases such as `open-class`. This can be due to the simplicity and cleanliness of the text data, indicating that the differences between two tokenizers could emerge due to their different tokenization rules.

Evaluation between Moses and NLTK

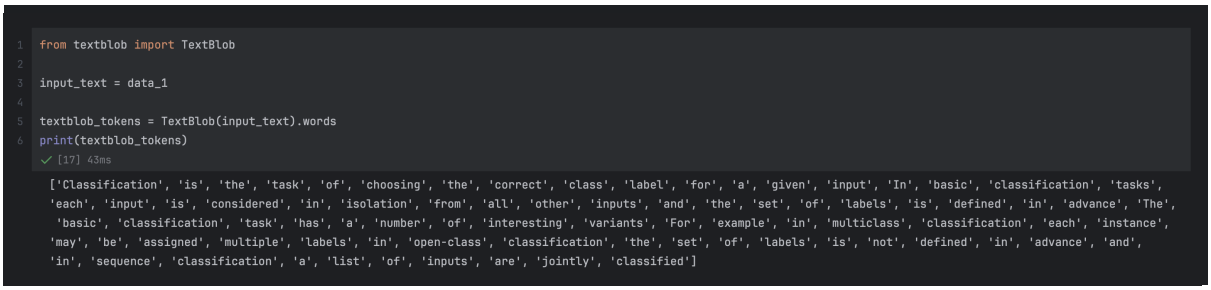
In short, the observation has proven that `MosesTokenizer` is not better or worse than NLTK library tokenizer, but they are just designed for different purposes. The key difference is on their focus of application, where NLTK tokenizer is for general purpose NLP tasks, and `MosesTokenizer` can be better in terms of its linguistic rules for machine translation, which helps to preserve more meaningful structures such as hyphenated words when handling complex text data (Guskov, 2023). Furthermore, it also supports additional detokenization, which enables it to be applied in tasks where text construction is needed. However, `MosesTokenizer` is heavier than NLTK's tokenizer as it requires additional installation. Hence, it might not provide extra benefits with increasing complexity in handling basic text classification tasks.

A.5.2 Eu Jun Hong (TP068580): TextBlob Library

Word Tokenisation Implementation

Figure 16

Word Tokenisation using TextBlob Library



```
from textblob import TextBlob

input_text = data_1

textblob_tokens = TextBlob(input_text).words
print(textblob_tokens)
```

```
✓ [17] 43ms
['Classification', 'is', 'the', 'task', 'of', 'choosing', 'the', 'correct', 'class', 'label', 'for', 'a', 'given', 'input', 'In', 'basic', 'classification', 'tasks',
'each', 'input', 'is', 'considered', 'in', 'isolation', 'from', 'all', 'other', 'inputs', 'and', 'the', 'set', 'of', 'labels', 'is', 'defined', 'in', 'advance', 'The',
 'basic', 'classification', 'task', 'has', 'a', 'number', 'of', 'interesting', 'variants', 'For', 'example', 'in', 'multiclass', 'classification', 'each', 'instance',
'may', 'be', 'assigned', 'multiple', 'labels', 'in', 'open-class', 'classification', 'the', 'set', 'of', 'labels', 'is', 'not', 'defined', 'in', 'advance', 'and',
'in', 'sequence', 'classification', 'a', 'list', 'of', 'inputs', 'are', 'jointly', 'classified']
```

Differences and Evaluation between TextBlob and NLTK

By comparison, TextBlob and NLTK provides minor differences because TextBlob is developed on top of NLTK. One of the main differences between both tokenization tools is the preprocessing philosophy. NLTK is very strict in tokenization especially keeping the structure of original sentence, which also means that punctuations are treated as unique tokens. On the other hand, TextBlob abstracts the tokenization process by integrating it with automatic text cleaning which includes filtering out all punctuations such as commas and remains the actual words only. For an example, TextBlob removes the `.` of the final sentence in the output and ends the output with `classified` word, while NLTK on the other hand splits `classified` and `.` as separate tokens. Hence, NLTK provide a better choice if a need of full control over tokenization, taggers and evaluation, while TextBlob provides better choice in simple POS tagging, lemmatization and sentiment (Ccoya & Pinto, 2023).

By comparison in the goal of accessible syntax and automatic punctuation removal, TextBlob is better than NLTK as it skips the filtering stage of manually removing punctuations to reduce dataset noise and dimensionality. Additionally, TextBlob is suitable for beginner projects and tasks that need to perform for a fast check as it is simple to implement in coding environments and focuses on general tasks such as translation, sentiments and noun phrases (Gichere, 2023). On the other hand, TextBlob is worst than NLTK in tokenization flexibility. Unlike NLTK which involves large amounts of toolkits like Treebank, Regexp or Tweet tokenizers, TextBlob does not provide customization options because it acts as a simplified wrapper which is suitable for beginners and simple tasks only.

A.5.3 Lim Beng Rhui (TP068495): Gensim Library

Word Tokenisation Implementation

Written by Radim Rehurek, the Gensim library provides different functions and utilities to perform NLP-related tasks (Navlani, 2022). To evaluate its effectiveness, tokenisation is performed using Gensim, as shown in Figure 17, to compare its difference with the NLTK approach implemented by the group.

Figure 17

Word Tokenisation using Gensim Library

```
# --- Remember to run "pip install gensim" to allow the code to run. ---

# Import gensim library
from gensim.utils import simple_preprocess

# Extract data
input_text = data_1

# Perform tokenization using gensim
gensim_tokens = simple_preprocess(input_text)

# Output results
print(gensim_tokens)
✓ [7] 135ms
['classification', 'is', 'the', 'task', 'of', 'choosing', 'the', 'correct', 'class', 'label', 'for', 'given', 'input', 'in',
'basic', 'classification', 'tasks', 'each', 'input', 'is', 'considered', 'in', 'isolation', 'from', 'all', 'other', 'inputs',
'and', 'the', 'set', 'of', 'labels', 'is', 'defined', 'in', 'advance', 'the', 'basic', 'classification', 'task', 'has',
'number', 'of', 'interesting', 'variants', 'for', 'example', 'in', 'multiclass', 'classification', 'each', 'instance', 'may',
'be', 'assigned', 'multiple', 'labels', 'in', 'open', 'class', 'classification', 'the', 'set', 'of', 'labels', 'is', 'not',
'defined', 'in', 'advance', 'and', 'in', 'sequence', 'classification', 'list', 'of', 'inputs', 'are', 'jointly', 'classified']
```

Difference between Gensim and NLTK

The `simple_preprocess()` function in Gensim automatically converts all the word tokens to lowercase and removes all punctuations during tokenisation, while NLTK's `word_tokenize()` function preserves the case of all word tokens and treats punctuations as individual tokens. Also, in default, Gensim removes all single-character words, such as `a` and `I`, to minimise any less meaningful tokens. Using the first sentence from `Data_1.txt` as an example, the differences between both approaches are highlighted as below:

```
NLTK    : ['Classification', 'is', 'the', 'task', 'of', 'choosing', 'the',
           'correct', 'class', 'label', 'for', 'a', 'given', 'input', '.', …]

Gensim  : ['classification', 'is', 'the', 'task', 'of', 'choosing', 'the',
           'correct', 'class', 'label', 'for', 'given', 'input', …]
```

Evaluation between Gensim and NLTK

To determine whether Gensim or NLTK is more appropriate to be implemented, it is crucial to evaluate them based on the objectives of the NLP task being performed:

- **Text classification and feature extraction**

  Since the `simple_preprocess()` function from Gensim produces cleaner outputs, where all the word tokens are standardised into lowercase letters without punctuations, this makes Gensim more suitable to be used for preprocessing data related to text classification tasks. As text classification tasks rely on vectorisation methods like TF-IDF, having clean and consistent word tokens minimises the preprocessing effort required to reduce feature complexity in these methods, thereby allowing classification models to focus on meaningful keywords and yield more accurate results (Nafis & Awang, 2019).

- **Linguistic analysis**

  For linguistic tasks like POS tagging and syntactic parsing, the NLTK approach outperforms Gensim as NLTK treats punctuation marks as separate tokens. Implementing Gensim's `simple_preprocess()` function will limit the models' ability in performing accurate linguistic analysis as the removal of punctuations, such as full stops (`.`) that represents sentence boundary, leads to the loss of contextual meaning and structural information of sentences (Min et al., 2025). This negatively impacts the models' ability when analysing grammatical relationships and contextual meanings behind sentences.

Therefore, for the supervised text classification task in Part B, the Gensim tokeniser is a more suitable approach as Gensim produces cleaner and more focused outputs, hence improving the predictive models' ability to capture meaningful keywords and boost its performance.

A.5.4 Ng Xuan Jack (TP067678): spaCy Library

Word Tokenisation Implementation

spaCy was implemented as an alternative approach to perform tokenisation on `Data_1.txt`'s sentence, as shown in Figure 18.

Figure 18

Word Tokenisation using spaCy Library

```
# Import library
import spacy

# Load English model
nlp = spacy.load("en_core_web_sm")

# Extract text
text = data_1

# Process tokenization
doc = nlp(text)
tokens = [token.text for token in doc]

# Remove stopwords and punctuation
filtered_tokens = [token.text for token in doc if not token.is_stop and not token.is_punct]

# Output results
print("Tokens:")
print(tokens)

print("\nFiltered Tokens:")
print(filtered_tokens)
```

✓ [9] 1s 369ms

```
Tokens:
['Classification', 'is', 'the', 'task', 'of', 'choosing', 'the', 'correct', 'class', 'label', 'for', 'a',
'given', 'input', '.', 'In', 'basic', '\n', 'classification', 'tasks', ',', 'each', 'input', 'is', 'considered',
 'in', 'isolation', 'from', 'all', 'other', 'inputs', ',', 'and', 'the', 'set', 'of', 'labels', 'is', 'defined',
 'in', 'advance', '.', 'The', 'basic', 'classification', 'task', 'has', 'a', 'number', 'of', 'interesting',
'variants', '.', 'For', 'example', ',', 'in', 'multiclass', 'classification', ',', 'each', 'instance', 'may',
'be', 'assigned', 'multiple', 'labels', ';', 'in', 'open', '-', 'class', 'classification', ',', 'the', 'set',
'of', 'labels', 'is', 'not', 'defined', 'in', 'advance', ';', 'and', 'in', 'sequence', 'classification', ',',
'a', 'list', 'of', 'inputs', 'are', 'jointly', 'classified', '.']

Filtered Tokens:
['Classification', 'task', 'choosing', 'correct', 'class', 'label', 'given', 'input', 'basic', '\n',
'classification', 'tasks', 'input', 'considered', 'isolation', 'inputs', 'set', 'labels', 'defined', 'advance',
'basic', 'classification', 'task', 'number', 'interesting', 'variants', 'example', 'multiclass',
'classification', 'instance', 'assigned', 'multiple', 'labels', 'open', 'class', 'classification', 'set',
'labels', 'defined', 'advance', 'sequence', 'classification', 'list', 'inputs', 'jointly', 'classified']
```

Differences and Evaluation between spaCy and NLTK

Unlike the group's approach which utilised the `word_tokenize()` function from NLTK, I've implemented spaCy to separate words and punctuations effectively while providing additional linguistic features like part-of-speech tags and dependency parsing to contrast with NLTK. Moreover, Neumann et al. (2019) further indicates that spaCy applies advanced pipeline-based tokenisation method that promises rule-based and linguistically informed which would provide diverse insights when compared with NLTK.

Conversely, NLTK performs well in basic tokenisation due to lightweight, making it suitable for simple preprocessing tasks (Bird et al., 2009). Additionally, NLTK successfully separates punctuation and maintains sentence structure to improve consistency. Nonetheless, spaCy offers more robust and production-ready solution handles edge cases like contractions and named entities more precisely through its pre-trained models (Omar et al., 2022).

Furthermore, spaCy demonstrates effective tokenisation by separating words, removing stop words and omitting punctuation. However, minor noise like newline characters was observed and removed through additional filtering (`token.is_space`). This ensures the importance of refining preprocessing steps for meaningful tokens for downstream NLP tasks (Hickman et al., 2022).

As for performance, spaCy is generally faster and more efficient when processing large datasets as it's optimised in Python. Thus, spaCy integrates tokenisation with other NLP tasks in a single pipeline, reducing the need for multiple libraries but requires larger memory usage and advance setup.

Overall, spaCy can be considered a better approach for real-world applications due to its scalability, speed and rich linguistic features. Nevertheless, for academic or smaller-scale tasks, NLTK remains more practical and sufficient choice. Therefore, two approaches are not better or worse but differ in complexity, efficiency and intended use cases.

Part B: Supervised Text Classification in NLP

This section outlines the different tasks involved when performing text classification.

B.1 Dataset Selection and Exploratory Data Analysis

B.1.1 Dataset Acquisition and Justification

Due to the uncontrollable increase in unsolicited and malicious messages, modern communication systems treat spam detection as a critical task to protect users from phishing attacks, financial fraud and privacy breaches (Patil, 2025). With the emergence of the field of natural language processing (NLP), researchers have employed numerous machine learning (Yadav & Kaur, 2023) and deep learning techniques (Tusher et al., 2025) to accurately identify spam messages, where one of the most prominent datasets used for spam detection is the SMS Spam Collection from the UCI Machine Learning Repository. Created by Almeida et al. (2011), the dataset is accessible via the URL:

```
SMS Spam Collection Dataset Link:
https://archive.ics.uci.edu/dataset/228/sms+spam+collection
```

The SMS Spam Collection dataset, with its attributes outlined in Table 3, contains a total of 5574 SMS messages that are collected from real-world sources, where each SMS is labelled as either *spam* (unwanted promotional messages) or *ham* (legitimate messages). As the medium-sized dataset offers a clean structure and well-defined attributes, along with its widespread adoption in prior research (Shaikh et al., 2025), this justifies the suitability of the dataset for performing NLP-related preprocessing and classification tasks while providing insights to improve the performance of spam filtering systems implemented in real-world messaging platforms.

Table 3

Attributes of the SMS Spam Collection Dataset

Attribute	Type	Description
`label`	Categorical (Binary)	Class label indicating whether a message is *spam* (unwanted promotional content) or *ham* (legitimate message)
`message`	Text	Raw SMS text content containing user messages collected from real-world sources

B.1.2 Exploratory Data Analysis

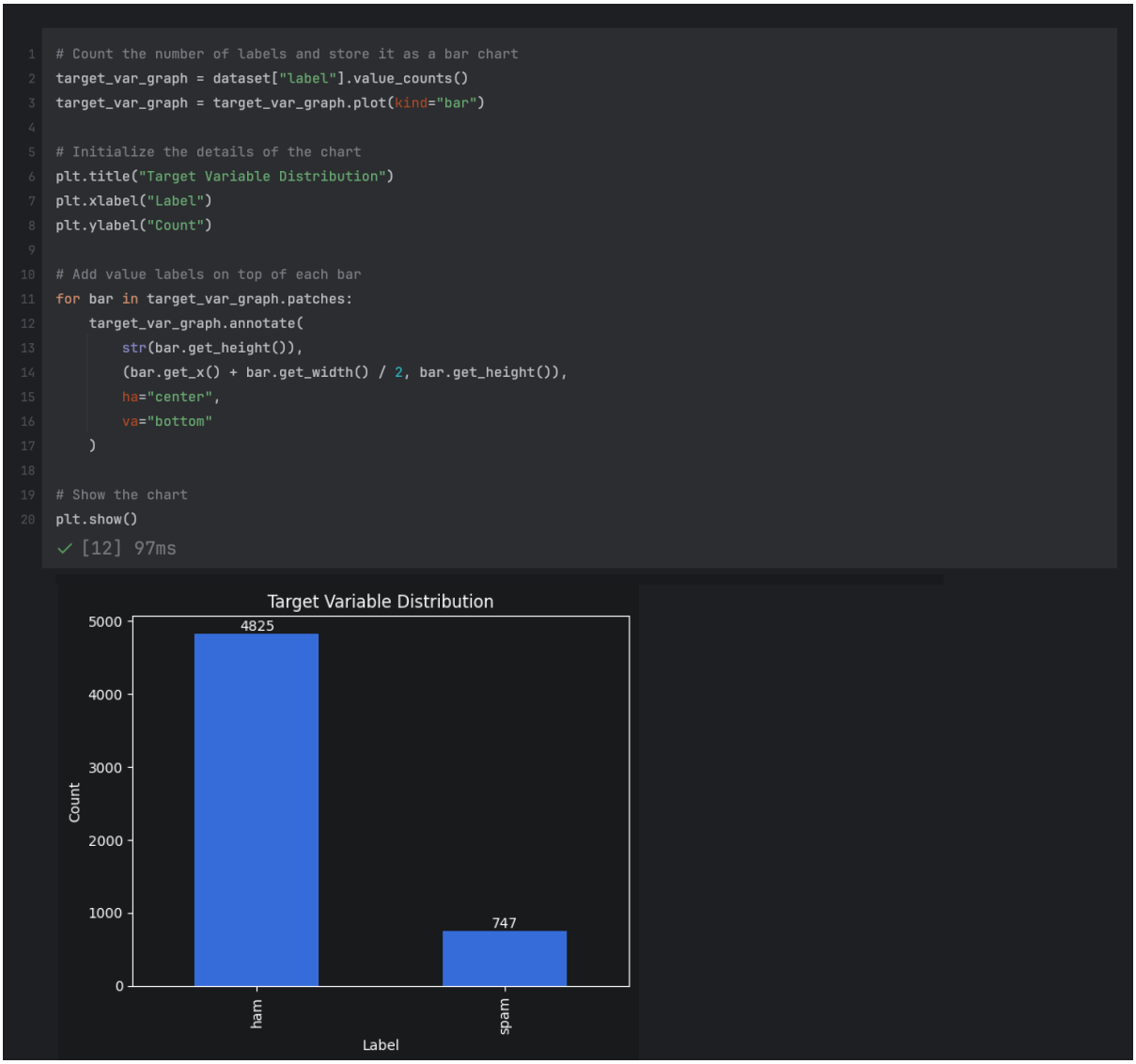
Before performing data preprocessing steps, an initial exploratory data analysis (EDA) is performed to understand the values and distributions of different attributes.

Target Variable Distribution

Figure 19 shows the class distribution of target variables, where it is observed that the number of *hams* (4,825) is significantly more than the number of *spams* (747). This indicates that class imbalance exists in the dataset, which must be tackled to minimise bias towards the majority class during model training. To mitigate this issue, the SMOTE technique will be used to perform oversampling on the *spam* class and improve model robustness (Airlangga, 2024).

Figure 19

Target Variable Distribution



Character Count and Sentence Count Distribution

The density plots in Figure 20 demonstrate that the *ham* class generally has a shorter length than the *spam* class in terms of character and sentence counts. Looking at the distinction between both classes, two additional attributes named `character_count` and `sentence_count` will be engineered and added to the dataset to improve model performance, aligning with the preprocessing steps from Johari et al. (2025) who also integrate character counts into prediction.

Figure 20

Character Count and Sentence Count Distribution

```
# First, calculate the character count and the sentence count
dataset["character_count"] = dataset["message"].apply(len)
dataset["sentence_count"] = dataset["message"].apply(
    lambda text: len(nltk.sent_tokenize(text))
)

# Display the dataset to show successful modification
print(dataset.head())

# Divide the data based on ham and spam
ham = dataset[dataset["label"] == "ham"]
spam = dataset[dataset["label"] == "spam"]

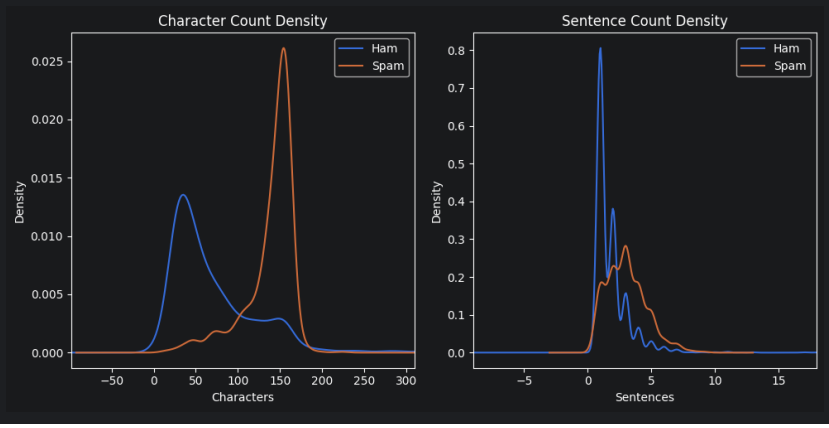
# Set up the figure size
plt.figure(figsize=(10,5))

# Plot for character counts
plt.subplot(1, 2, 1)
ham["character_count"].plot(kind="kde", label="Ham")
spam["character_count"].plot(kind="kde", label="Spam")
plt.title("Character Count Density")
plt.xlabel("Characters")
plt.xlim(dataset["character_count"].min() - 100, dataset["character_count"].max() - 600)    # This is to limit the x-axis to the data range
plt.legend()

# Plot for sentence counts
plt.subplot(1, 2, 2)
ham["sentence_count"].plot(kind="kde", label="Ham")
spam["sentence_count"].plot(kind="kde", label="Spam")
plt.title("Sentence Count Density")
plt.xlabel("Sentences")
plt.xlim(dataset["sentence_count"].min() - 10, dataset["sentence_count"].max() - 20)      # This is to limit the x-axis to the data range
plt.legend()

# Display the plots
plt.tight_layout()
plt.show()
✓ [20] 275ms
```

	label	message	character_count	sentence_count
0	ham	Go until jurong point, crazy.. Available only ...	111	2
1	ham	Ok lar... Joking wif u oni...	29	2
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	155	2
3	ham	U dun say so early hor... U c already then say...	49	1
4	ham	Nah I don't think he goes to usf, he lives aro...	61	1

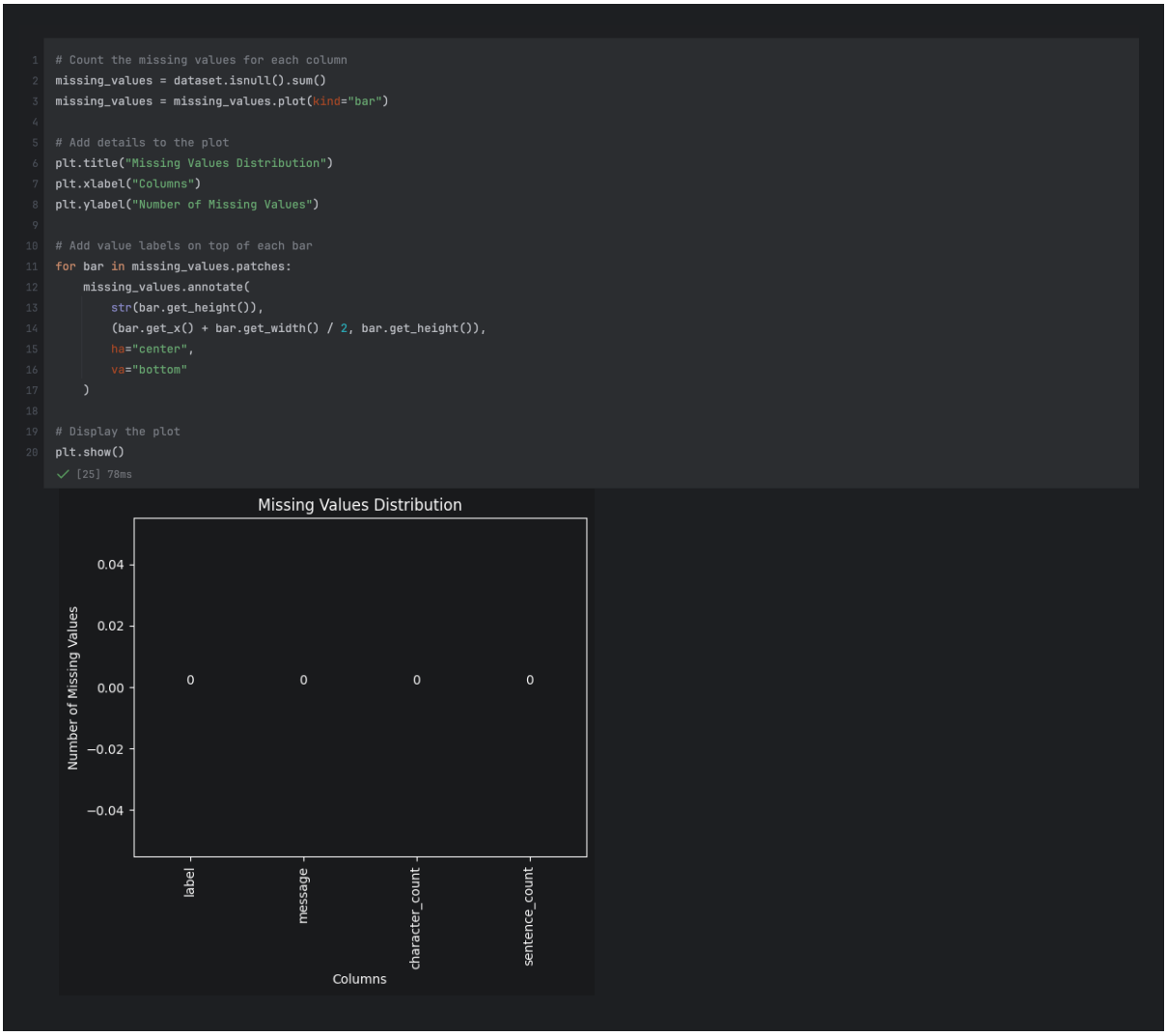


Missing Value Distribution

Based on the missing values distribution graph in Figure 21, it is observed that the columns in the dataset, including `label`, `message`, `character_count` and `sentence_count`, do not contain missing values. Hence, row removal and data imputation will not be performed.

Figure 21

Missing Value Distribution



Correlation Matrix

The correlation matrix in Figure 22 shows that the strongest relationship between attributes is moderately positive (0.63), which does not exceed the rule-of-thumb threshold of 0.8 that indicates severe multicollinearity issues in the dataset (Imdadullah et al., 2016). Therefore, the dataset does not contain any redundant attributes that shall be removed, and all features will be retained for further preprocessing tasks.

Figure 22

Correlation Matrix

```
# First convert target variable to binary (so that correlation can be shown)
dataset["label_binary"] = dataset["label"].map({"ham": 0, "spam": 1})

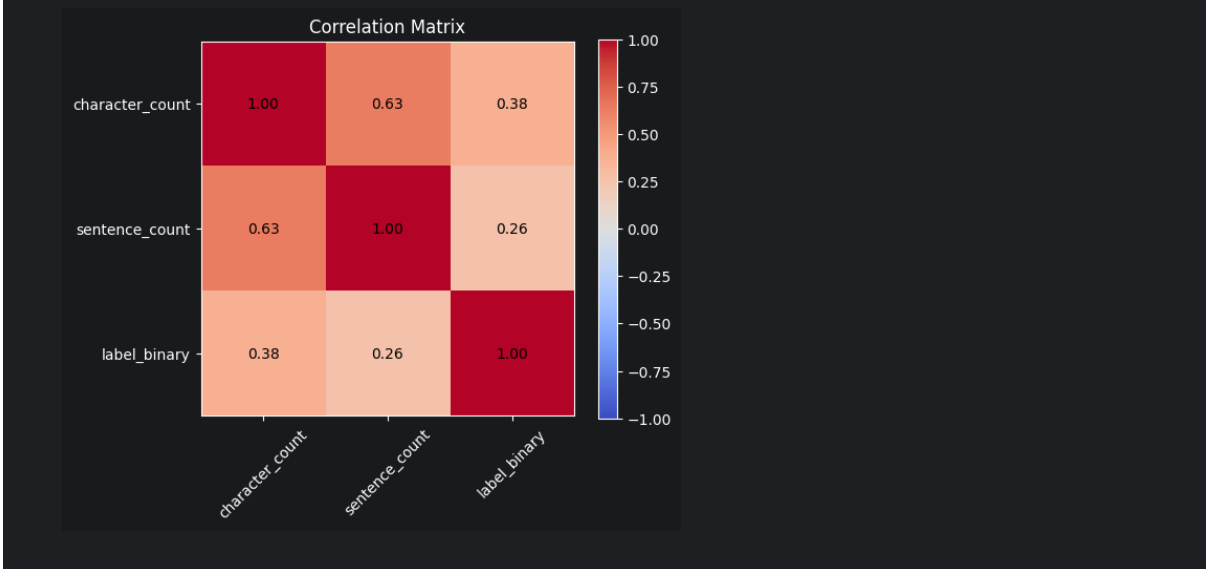
# Select the numerical values and the target variable
numeric_data = dataset[["character_count", "sentence_count", "label_binary"]]

# Compute the correlation matrix
correlation_matrix = numeric_data.corr()

# Plot the matrix
plt.figure(figsize=(6,5))
plt.imshow(correlation_matrix, cmap="coolwarm", vmin=-1, vmax=1)
plt.xticks(range(len(correlation_matrix.columns)), correlation_matrix.columns, rotation=45)
plt.yticks(range(len(correlation_matrix.columns)),  correlation_matrix.columns)
plt.title("Correlation Matrix")
plt.colorbar()

# Add correlation values inside cells
for i in range(len(correlation_matrix)):
    for j in range(len(correlation_matrix)):
        plt.text(j, i,
                 f"{correlation_matrix.iloc[i, j]:.2f}",
                 ha="center",
                 va="center",
                 color="black")

# Display the plot
plt.tight_layout()
plt.show()
✓ [28] 132ms
```



B.1.3 Machine Learning Model Selection

Based on the review of existing literatures, a wide range of machine learning models are implemented by researchers to perform text classification tasks on the SMS Spam Collection dataset. Hence, the classification algorithms that will be implemented are highlighted in Table 4.

Table 4

Classification Algorithms Selected for Project

Algorithm	Reference	Methodology / Preprocessing Steps	Key Findings	Performance
Naïve Bayes classifier	Miller (2025)	• Filtered dataset to 50 spams and 50 hams. • Data cleaning procedures are not outlined.	• Machine learning techniques shall be incorporated with experts (i.e., cybersecurity professionals) to maximise detection accuracy.	• Accuracy: 91.98%
Support Vector Machine	Bansode & Pingle (2025)	• Preprocessed the dataset by: ○ Stop-word and punctuation removal ○ Tokenization ○ TF-IDF vectorization • Dataset was split in an 80:20 ratio	• Combining probabilistic (Naïve Bayes) and margin-based classifiers (SVM) yields higher classification performance	• Accuracy: 98.91% • Precision: 98.47%
Decision Tree	Johari et al. (2025)	• Only retain messages and target variable • Remove rows with missing values • Trimmed white spaces around text • Stop-word removal using NLTK	• Decision trees are highly interpretable and can capture non-linear relationships • However, decision trees are inefficient when handling sparsely distributed text data	• Accuracy: 96.86% • Recall: 99%
Logistic Regression	Garg et al. (2022)	• Performed label encoding • Removed stop words • Stemming to reduce vocabulary space • Vectorization using TF-IDF	• Deep learning techniques can be incorporated to improve performance but requires large datasets • Traditional models like logistic regression remain lightweight and effective for text classification tasks	• Performance metrics not reported.

B.1.4 Data Preprocessing

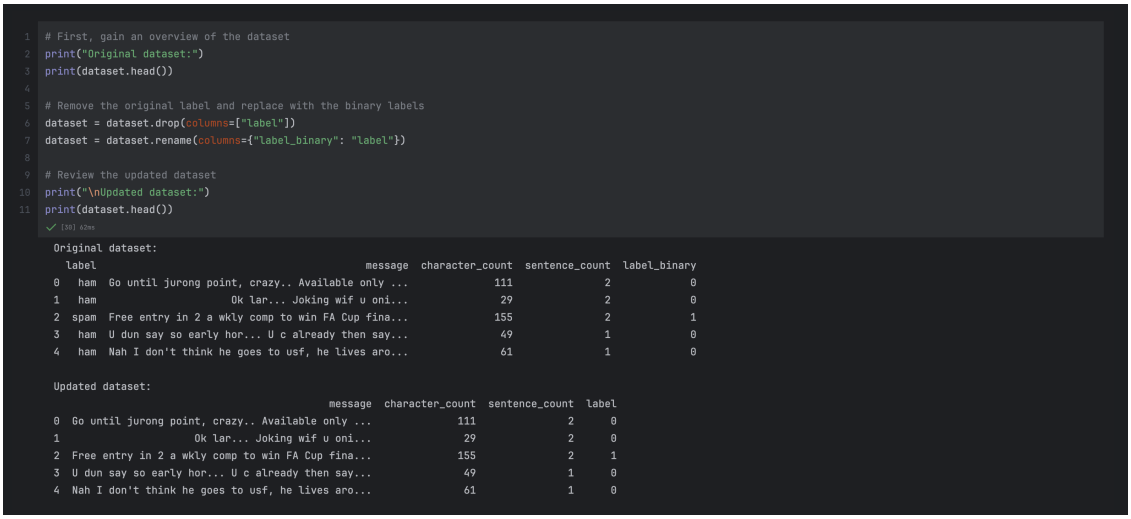
The preprocessing steps performed to clean the dataset include:

- **Label encoding**

  During the initial EDA, the target variable `label` is converted into binary values, where `ham = 0` and `spam = 1`. Hence, the `label_binary` column replaces the `label` column so that the target variable can be used on different machine learning algorithms.

Figure 23

Label Encoding for Dataset



```
# First, gain an overview of the dataset
print("Original dataset:")
print(dataset.head())

# Remove the original label and replace with the binary labels
dataset = dataset.drop(columns=["label"])
dataset = dataset.rename(columns={"label_binary": "label"})

# Review the updated dataset
print("\nUpdated dataset:")
print(dataset.head())
```

✓ [38] 62ms

```
Original dataset:
  label                                            message  character_count  sentence_count  label_binary
0   ham  Go until jurong point, crazy.. Available only ...              111              2             0
1   ham                      Ok lar... Joking wif u oni...               29              2             0
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...              155              2             1
3   ham  U dun say so early hor... U c already then say...               49              1             0
4   ham  Nah I don't think he goes to usf, he lives aro...               61              1             0

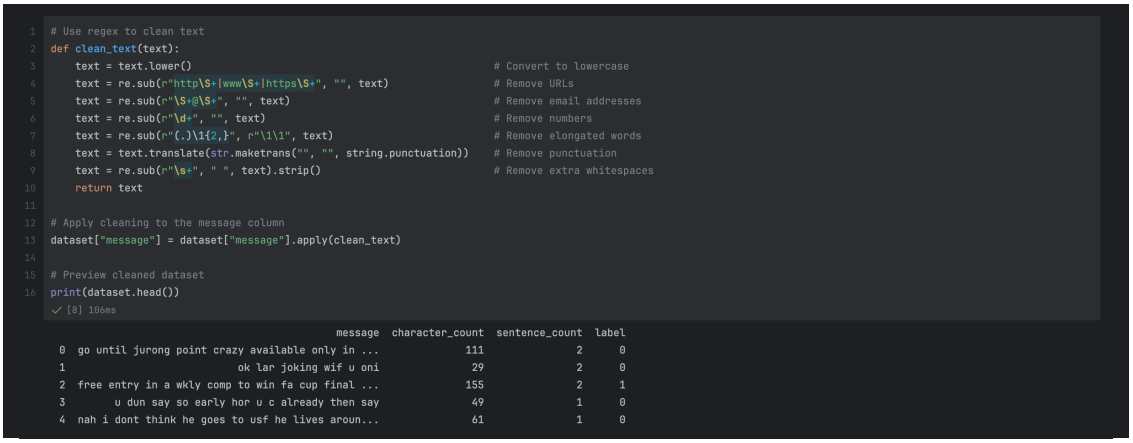
Updated dataset:
                                             message  character_count  sentence_count  label
0  Go until jurong point, crazy.. Available only ...              111              2      0
1                      Ok lar... Joking wif u oni...               29              2      0
2  Free entry in 2 a wkly comp to win FA Cup fina...              155              2      1
3  U dun say so early hor... U c already then say...               49              1      0
4  Nah I don't think he goes to usf, he lives aro...               61              1      0
```

- **Text cleaning**

  The `message` column is converted to lowercase with information like URL, email addresses, numbers and punctuations removed to facilitate subsequent tasks.

Figure 24

Text Cleaning on Dataset



```
# Use regex to clean text
def clean_text(text):
    text = text.lower()                                                  # Convert to lowercase
    text = re.sub(r"http\S+|www\S+|https\S+", "", text)                  # Remove URLs
    text = re.sub(r"\S+@\S+", "", text)                                  # Remove email addresses
    text = re.sub(r"\d+", "", text)                                      # Remove numbers
    text = re.sub(r"(.)\1{2,}", r"\1\1", text)                           # Remove elongated words
    text = text.translate(str.maketrans("", "", string.punctuation))    # Remove punctuation
    text = re.sub(r"\s+", " ", text).strip()                             # Remove extra whitespaces
    return text

# Apply cleaning to the message column
dataset["message"] = dataset["message"].apply(clean_text)

# Preview cleaned dataset
print(dataset.head())
```

✓ [8] 106ms

```
                                             message  character_count  sentence_count  label
0  go until jurong point crazy available only in ...              111              2      0
1                        ok lar joking wif u oni               29              2      0
2  free entry in a wkly comp to win fa cup final ...              155              2      1
3        u dun say so early hor u c already then say               49              1      0
4  nah i dont think he goes to usf he lives aroun...               61              1      0
```

- **Tokenisation and stop word removal**

  Using NLTK, the words in `message` are tokenised with stop words removed.

Figure 25

Tokenisation and Stop Word Removal on Dataset

```
# Retrieve the English stop words from NLTK's corpus
stop_words = set(stopwords.words("english"))

# A function to perform tokenization and stop word removal in one shot
def tokenize_and_remove_stopwords(text):
    tokens = word_tokenize(text)                                    # Perform tokenization
    clean_tokens = [word for word in tokens if word not in stop_words]      # Remove stop words if it exist in NLTK's stopword corpus
    return clean_tokens

# Apply to message column
dataset["tokens"] = dataset["message"].apply(tokenize_and_remove_stopwords)

# Preview results
print(dataset.head())
✓ [9] 261ms
```

```
                                             message  character_count  sentence_count  label                                             tokens
0  go until jurong point crazy available only in ...              111               2      0  [go, jurong, point, crazy, available, bugis, n...
1                            ok lar joking wif u oni               29               2      0                      [ok, lar, joking, wif, u, oni]
2  free entry in a wkly comp to win fa cup final ...              155               2      1  [free, entry, wkly, comp, win, fa, cup, final,...
3        u dun say so early hor u c already then say               49               1      0      [u, dun, say, early, hor, u, c, already, say]
4  nah i dont think he goes to usf he lives aroun...               61               1      0  [nah, dont, think, goes, usf, lives, around, t...
```

- **POS tagging and lemmatisation**

  POS tagging is performed to provide grammatical context for better lemmatization.

Figure 26

POS Tagging and Lemmatisation on Dataset

```
# Create a lemmatizer object
lemmatizer = WordNetLemmatizer()

# A function to convert tags from POS tagging to tags used by WordNetLemmatizer
def get_wordnet_pos(tag):
    if tag.startswith('J'):     return wordnet.ADJ
    elif tag.startswith('V'):   return wordnet.VERB
    elif tag.startswith('N'):   return wordnet.NOUN
    elif tag.startswith('R'):   return wordnet.ADV
    else:                       return wordnet.NOUN

# The function to return the lemmatized tokens
def lemmatize_with_pos(tokens):
    tagged = nltk.pos_tag(tokens)
    return [
        lemmatizer.lemmatize(word, get_wordnet_pos(tag)) for word, tag in tagged
    ]

# Perform lemmatizations (with POS tagging) on the dataset
dataset["lemmatized_tokens"] = dataset["tokens"].apply(lemmatize_with_pos)

# Display results
print(dataset[["tokens", "lemmatized_tokens"]].head())
✓ [12] 3s 380ms
```

```
                                              tokens                                   lemmatized_tokens
0  [go, jurong, point, crazy, available, bugis, n...  [go, jurong, point, crazy, available, bugis, n...
1                      [ok, lar, joking, wif, u, oni]                      [ok, lar, joking, wif, u, oni]
2  [free, entry, wkly, comp, win, fa, cup, final,...  [free, entry, wkly, comp, win, fa, cup, final,...
3      [u, dun, say, early, hor, u, c, already, say]      [u, dun, say, early, hor, u, c, already, say]
4  [nah, dont, think, goes, usf, lives, around, t...  [nah, dont, think, go, usf, life, around, though]
```

- **Train-test split**

  To prevent data leakage for subsequent preprocessing tasks, the dataset is split into 80:20.

Figure 27

Train-Test Split on Dataset

```
# Divide the attributes into text columns (must combine for TF-IDF), numerical columns and target variable
X_text = dataset["lemmatized_tokens"].apply(lambda x: " ".join(x))
X_num = dataset[["character_count", "sentence_count"]]
y = dataset["label"]

# Perform splitting based on the ratio 80:20
X_text_train, X_text_test, X_num_train, X_num_test, y_train, y_test = train_test_split(
    X_text,
    X_num,
    y,
    test_size=0.2,
    random_state=1029,
    stratify=y
)

# Show the distribution of data after split
print("Training set size:", len(X_text_train))
print("Test set size:", len(X_text_test))

print("\nClass distribution (Train):")
print(y_train.value_counts(normalize=True))     # normalize=True is to show percentage instead of absolute values
print("\nClass distribution (Test):")
print(y_test.value_counts(normalize=True))
✓ [21] 76ms

Training set size: 4457
Test set size: 1115

Class distribution (Train):
label
0    0.865829
1    0.134171
Name: proportion, dtype: float64

Class distribution (Test):
label
0    0.866368
1    0.133632
Name: proportion, dtype: float64
```

- **Vectorisation using TF-IDF**

  Vectorisation is then performed using TF-IDF with unigram and bigram representations and feature size limited to 5000 terms (Dasgupta & Mehr, 2024) to enhance efficiency.

Figure 28

TF-IDF Vectorisation on Dataset

```
# Create a vectorization object with limit set as 5000
tfidf = TfidfVectorizer(max_features=5000, ngram_range=(1,2))

# Perform TF-TDF vectorization on training set first, then apply the same transformation on testing set
X_train_tfidf = tfidf.fit_transform(X_text_train)
X_test_tfidf = tfidf.transform(X_text_test)

# Combine the TF-IDF features with numerical attributes
X_train_final = hstack([X_train_tfidf, X_num_train.values])
X_test_final = hstack([X_test_tfidf, X_num_test.values])

# Check the output shape
print("Train shape:", X_train_final.shape)
print("Test shape:", X_test_final.shape)
✓ [48] 115ms

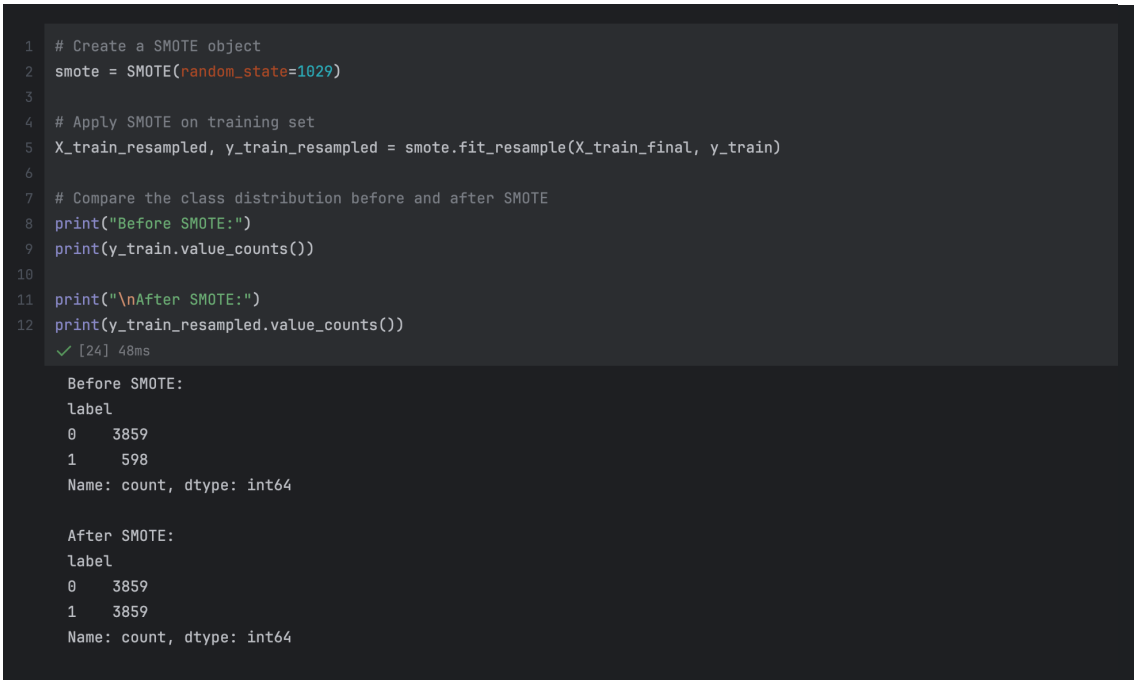
Train shape: (4457, 5002)
Test shape: (1115, 5002)
```

- **Handling class imbalance using SMOTE**

  Oversampling is lastly performed using the SMOTE technique to address class imbalance, minimise training bias, and improve classification performance (Airlangga, 2024).

Figure 29

Oversampling using SMOTE on Dataset



```
# Create a SMOTE object
smote = SMOTE(random_state=1029)

# Apply SMOTE on training set
X_train_resampled, y_train_resampled = smote.fit_resample(X_train_final, y_train)

# Compare the class distribution before and after SMOTE
print("Before SMOTE:")
print(y_train.value_counts())

print("\nAfter SMOTE:")
print(y_train_resampled.value_counts())
✓ [24] 48ms

Before SMOTE:
label
0    3859
1     598
Name: count, dtype: int64

After SMOTE:
label
0    3859
1    3859
Name: count, dtype: int64
```

B.2 Supervised Text Classification

Four classification algorithms, namely Naïve Bayes, support vector machine (SVM), decision tree and logistic regression, are implemented, with their corresponding Python code and output shown in Figures 30 to 34.

Figure 30

Multinomial Naïve Bayes Implemented on Dataset

```python
# Create the Naive Bayes model
nb = MultinomialNB()

# Train the model and perform prediction
nb.fit(X_train_resampled, y_train_resampled)
nb_predict = nb.predict(X_test_final)

# Evaluation
print("Baseline Accuracy:", accuracy_score(y_test, nb_predict))
print("Baseline Recall:", recall_score(y_test, nb_predict))
print("Baseline F1-Score:", f1_score(y_test, nb_predict))
print(classification_report(y_test, nb_predict))

# Define the parameters for hyperparameter tuning
nb_params = {
    "alpha": [0.01, 0.1, 0.5, 1.0, 2.0],
    "fit_prior": [True, False]
}

# Use grid search with 10-fold cross-validation to find the best params
nb_grid = GridSearchCV(
    MultinomialNB(),
    param_grid=nb_params,
    cv=10,
    scoring="f1",
    n_jobs=-1
)

# Use the tuned hyperparameter to retrain the model and output the best params and score (on training dataset)
nb_grid.fit(X_train_resampled, y_train_resampled)
print("Best Parameters:", nb_grid.best_params_)
print("Best CV F1-Score:", nb_grid.best_score_)

# Retrieve the best model, run on the testing dataset, and output results
best_nb = nb_grid.best_estimator_
best_nb_predict = best_nb.predict(X_test_final)
print("\nOptimized Test Accuracy:", accuracy_score(y_test, best_nb_predict))
print("Optimized Test Recall:", recall_score(y_test, best_nb_predict))
print("Optimized Test F1-Score:", f1_score(y_test, best_nb_predict))
print(classification_report(y_test, best_nb_predict))
```

✓ [49] 2s 305ms

```
Baseline Accuracy: 0.9533632286995516
Baseline Recall: 0.9664429530201343
Baseline F1-Score: 0.8470588235294118
              precision    recall  f1-score   support

           0       0.99      0.95      0.97       966
           1       0.75      0.97      0.85       149

    accuracy                           0.95      1115
   macro avg       0.87      0.96      0.91      1115
weighted avg       0.96      0.95      0.96      1115

Best Parameters: {'alpha': 0.01, 'fit_prior': True}
Best CV F1-Score: 0.9840755611165879

Optimized Test Accuracy: 0.9623318385650225
Optimized Test Recall: 0.9261744966442953
Optimized Test F1-Score: 0.8679245283018868
              precision    recall  f1-score   support

           0       0.99      0.97      0.98       966
           1       0.82      0.93      0.87       149

    accuracy                           0.96      1115
   macro avg       0.90      0.95      0.92      1115
weighted avg       0.97      0.96      0.96      1115
```

Figure 31

Kernel-Based SVM Implemented on Dataset

```python
# Create the SVM model
svm = SVC()

# Train the model and perform prediction
svm.fit(X_train_resampled, y_train_resampled)
svm_predict = svm.predict(X_test_final)

# Output baseline metrics
print("Baseline Accuracy:", accuracy_score(y_test, svm_predict))
print("Baseline Recall:", recall_score(y_test, svm_predict))
print("Baseline F1-Score:", f1_score(y_test, svm_predict))
print(classification_report(y_test, svm_predict))

# List down the parameters to be tuned
svm_params = {
    "C": [0.1, 1.0, 10, 100, 500, 1000],
    "kernel": ["linear", "rbf", "poly"],
    "gamma": ["scale", "auto"]
}

# Perform hyperparameter tuning using Grid Search
svm_grid = GridSearchCV(
    SVC(),
    param_grid=svm_params,
    cv=10,
    scoring="f1",
    n_jobs=-1
)

# Train the SVM on the training dataset and output performance (based on training)
svm_grid.fit(X_train_resampled, y_train_resampled)
print("Best Parameters:", svm_grid.best_params_)
print("Best CV F1-Score:", svm_grid.best_score_)

# Use the best hyperparameter to test on the training set and output the results
best_svm = svm_grid.best_estimator_
best_svm_predict = best_svm.predict(X_test_final)
print("\nOptimized Test Accuracy:", accuracy_score(y_test, best_svm_predict))
print("Optimized Test Recall:", recall_score(y_test, best_svm_predict))
print("Optimized Test F1-Score:", f1_score(y_test, best_svm_predict))
print(classification_report(y_test, best_svm_predict))
```

✓ [53] 46m 34s

```
Baseline Accuracy: 0.8286995515695067
Baseline Recall: 0.9060402684563759
Baseline F1-Score: 0.5856832971800434
              precision    recall  f1-score   support

           0       0.98      0.82      0.89       966
           1       0.43      0.91      0.59       149

    accuracy                           0.83      1115
   macro avg       0.71      0.86      0.74      1115
weighted avg       0.91      0.83      0.85      1115

Best Parameters: {'C': 1000, 'gamma': 'auto', 'kernel': 'rbf'}
Best CV F1-Score: 0.9949556299025935

Optimized Test Accuracy: 0.9829596412556054
Optimized Test Recall: 0.8926174496644296
Optimized Test F1-Score: 0.9333333333333333
              precision    recall  f1-score   support

           0       0.98      1.00      0.99       966
           1       0.98      0.89      0.93       149

    accuracy                           0.98      1115
   macro avg       0.98      0.94      0.96      1115
weighted avg       0.98      0.98      0.98      1115
```

Figure 32

Linear SVM Implemented on Dataset

```python
# Create the linear SVM model
lin_svm = LinearSVC()

# Train the model and perform prediction
lin_svm.fit(X_train_resampled, y_train_resampled)
lin_svm_predict = lin_svm.predict(X_test_final)

# Output baseline metrics
print("Baseline Accuracy:", accuracy_score(y_test, lin_svm_predict))
print("Baseline Recall:", recall_score(y_test, lin_svm_predict))
print("Baseline F1-Score:", f1_score(y_test, lin_svm_predict))
print(classification_report(y_test, lin_svm_predict))

# List down the parameters to be tuned
lin_svm_params = {
    "C": [0.1, 1, 10, 100, 500, 1000, 5000, 10000],
    "loss": ["squared_hinge"]
}

# Perform hyperparameter tuning using Grid Search
lin_svm_grid = GridSearchCV(
    LinearSVC(),
    param_grid=lin_svm_params,
    cv=10,
    scoring="f1",
    n_jobs=-1
)

# Train the linear SVM on the training dataset and output performance (based on training)
lin_svm_grid.fit(X_train_resampled, y_train_resampled)
print("Best Parameters:", lin_svm_grid.best_params_)
print("Best CV F1-Score:", lin_svm_grid.best_score_)

# Use the best hyperparameter to test on the training set and output the results
best_lin_svm = lin_svm_grid.best_estimator_
best_lin_svm_predict = best_lin_svm.predict(X_test_final)
print("\nOptimized Test Accuracy:", accuracy_score(y_test, best_lin_svm_predict))
print("Optimized Test Recall:", recall_score(y_test, best_lin_svm_predict))
print("Optimized Test F1-Score:", f1_score(y_test, best_lin_svm_predict))
print(classification_report(y_test, best_lin_svm_predict))
```

✓ [65] 504ms

```
Baseline Accuracy: 0.9856502242152466
Baseline Recall: 0.912751677852349
Baseline F1-Score: 0.9444444444444444
              precision    recall  f1-score   support

           0       0.99      1.00      0.99       966
           1       0.98      0.91      0.94       149

    accuracy                           0.99      1115
   macro avg       0.98      0.95      0.97      1115
weighted avg       0.99      0.99      0.99      1115

Best Parameters: {'C': 5000, 'loss': 'squared_hinge'}
Best CV F1-Score: 0.9954897863486367

Optimized Test Accuracy: 0.9856502242152466
Optimized Test Recall: 0.9194630872483222
Optimized Test F1-Score: 0.9448275862068966
              precision    recall  f1-score   support

           0       0.99      1.00      0.99       966
           1       0.97      0.92      0.94       149

    accuracy                           0.99      1115
   macro avg       0.98      0.96      0.97      1115
weighted avg       0.99      0.99      0.99      1115
```

Figure 33

Decision Tree Implemented on Dataset

```
# Initialize the decision tree model
dt = DecisionTreeClassifier(random_state=1029)

# Train model on training dataset and predict model on testing dataset
dt.fit(X_train_resampled, y_train_resampled)
dt_predict = dt.predict(X_test_final)

# Output the baseline performance
print("Baseline Accuracy:", accuracy_score(y_test, dt_predict))
print("Baseline Recall:", recall_score(y_test, dt_predict))
print("Baseline F1-Score:", f1_score(y_test, dt_predict))
print(classification_report(y_test, dt_predict))

# Outline the list of parameters to be tuned
dt_params = {
    'max_depth': [1, 5, 10, 20, 25, 30, 50],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 5],
    'criterion': ['gini', 'entropy']
}

# Perform grid search to find for the best hyperparameter
dt_grid = GridSearchCV(
    DecisionTreeClassifier(random_state=1029),
    param_grid=dt_params,
    cv=10,
    scoring="f1",
    n_jobs=-1
)

# Use the best hyperparameter on the training dataset
dt_grid.fit(X_train_resampled, y_train_resampled)
print("Best Parameters:", dt_grid.best_params_)
print("Best CV F1-Score:", dt_grid.best_score_)

# Implement the best hyperparameter on the testing dataset and output results
best_dt = dt_grid.best_estimator_
best_dt_predict = best_dt.predict(X_test_final)
print("\nOptimized Test Accuracy:", accuracy_score(y_test, best_dt_predict))
print("Optimized Test Recall:", recall_score(y_test, best_dt_predict))
print("Optimized Test F1-Score:", f1_score(y_test, best_dt_predict))
print(classification_report(y_test, best_dt_predict))
✓ [50] 16s 25ms
```

```
Baseline Accuracy: 0.9587443946188341
Baseline Recall: 0.8791946308724832
Baseline F1-Score: 0.8506493506493507
              precision    recall  f1-score   support

           0       0.98      0.97      0.98       966
           1       0.82      0.88      0.85       149

    accuracy                           0.96      1115
   macro avg       0.90      0.93      0.91      1115
weighted avg       0.96      0.96      0.96      1115

Best Parameters: {'criterion': 'gini', 'max_depth': 25, 'min_samples_leaf': 1, 'min_samples_split': 2}
Best CV F1-Score: 0.9773554368961817

Optimized Test Accuracy: 0.9632286995515695
Optimized Test Recall: 0.87248322147651
Optimized Test F1-Score: 0.8637873754152824
              precision    recall  f1-score   support

           0       0.98      0.98      0.98       966
           1       0.86      0.87      0.86       149

    accuracy                           0.96      1115
   macro avg       0.92      0.92      0.92      1115
weighted avg       0.96      0.96      0.96      1115
```

Figure 34

Logistic Regression Implemented on Dataset

```python
# Initialize the logistic regression model
lr = LogisticRegression(max_iter=2000)

# Train the model using training dataset and run the model on testing dataset
lr.fit(X_train_resampled, y_train_resampled)
lr_predict = lr.predict(X_test_final)

# Output the baseline performance
print("Baseline Accuracy:", accuracy_score(y_test, lr_predict))
print("Baseline Recall:", recall_score(y_test, lr_predict))
print("Baseline F1-Score:", f1_score(y_test, lr_predict))
print(classification_report(y_test, lr_predict))

# Outline the list of hyperparameters to be tuned
lr_params = {
    "C": [0.01, 0.1, 1, 10, 100, 200, 500, 1000],
    "solver": ["newton-cg", "lbfgs", "liblinear"]
}

# Perform hyperparameter tuning using grid search
lr_grid = GridSearchCV(
    LogisticRegression(max_iter=2000),
    param_grid=lr_params,
    cv=10,
    scoring="f1",
    n_jobs=-1
)

# Apply the model on the training dataset and output performance (training)
lr_grid.fit(X_train_resampled, y_train_resampled)
print("Best Parameters:", lr_grid.best_params_)
print("Best CV F1-Score:", lr_grid.best_score_)

# Apply the best hyperparameters on the testing dataset and output results (testing)
best_lr = lr_grid.best_estimator_
best_lr_predict = best_lr.predict(X_test_final)
print("Optimized Test Accuracy:", accuracy_score(y_test, best_lr_predict))
print("Optimized Test Recall:", recall_score(y_test, best_lr_predict))
print("Optimized Test F1-Score:", f1_score(y_test, best_lr_predict))
print(classification_report(y_test, best_lr_predict))
```

✓ [51] 3s 636ms

```
Baseline Accuracy: 0.9721973094170404
Baseline Recall: 0.9261744966442953
Baseline F1-Score: 0.8990228013029316
              precision    recall  f1-score   support

           0       0.99      0.98      0.98       966
           1       0.87      0.93      0.90       149

    accuracy                           0.97      1115
   macro avg       0.93      0.95      0.94      1115
weighted avg       0.97      0.97      0.97      1115

Best Parameters: {'C': 500, 'solver': 'liblinear'}
Best CV F1-Score: 0.9954881427629987
Optimized Test Accuracy: 0.9865470852017937
Optimized Test Recall: 0.9261744966442953
Optimized Test F1-Score: 0.9484536082474226
              precision    recall  f1-score   support

           0       0.99      1.00      0.99       966
           1       0.97      0.93      0.95       149

    accuracy                           0.99      1115
   macro avg       0.98      0.96      0.97      1115
weighted avg       0.99      0.99      0.99      1115
```

B.3 Hyperparameter Selection and Optimisation

B.3.1 Hyperparameters in Models

The hyperparameters involved in the machine learning models trained are detailed in Table 5.

Table 5

Hyperparameters Involved in Models

Model	Hyperparameter	Explanation
Naïve Bayes	`alpha`	Smoothing parameter (Laplace or Lidstone smoothing) to prevent zero probability when encountering unseen words.
	`fit_prior`	Determine if the model should learn the class prior probabilities automatically from training data to handle class imbalance.
Support Vector Machine (Kernel)	`C`	Regularization parameter for SVC models that controls how well does the model fit to the training data.
	`gamma`	Coefficient to control the influence of individual training samples towards the SVM model.
	`kernel`	The kernel type to be used in the algorithm to capture either linear or non-linear relationships between attributes.
Support Vector Machine (Linear)	`C`	Regularization parameter to control how well the model fits to the training data.
	`loss`	Loss function that determines how errors in classification are penalised during model optimisation.
Decision Tree	`criterion`	Function to measure the quality of node splits during tree construction.
	`max_depth`	Maximum depth of the decision tree to balance training dataset fitting while preventing overfitting.
	`min_samples_leaf`	Minimum number of samples that must remain in a leaf node during splitting.
	`min_samples_split`	Minimum number of samples required before an internal node is split.
Logistic Regression	`C`	Regulation parameter that balances between fitting of training data and overfitting.
	`solver`	Algorithm to be used during optimization (when finding the best coefficient for logistic regression models).

Note. Compiled and adapted from Pedregosa et al. (2011).

B.3.2 Hyperparameter Optimisation Process

To improve the performance of the classification models, hyperparameter tuning is performed using `GridSearchCV` from `scikit-learn` due to its exhaustive coverage of different parameter combinations to yield optimal results, along with its ease of implementation (Equbal, 2024) compared to other approaches like random search and Bayesian optimization. In this project, the implementation of `GridSearchCV` is as follows:

- **Identification of hyperparameters for optimization**

  The key hyperparameters for each model are identified with their searching range defined based on existing literature (Bhuva, 2025; Borade, 2026; Brownlee, 2020; Nguyen, 2022).

Figure 35

Hyperparameter Configuration for Different Models

Multinomial Naïve Bayes:

```
nb_params = {
    "alpha": [0.01, 0.1, 0.5, 1.0, 2.0],
    "fit_prior": [True, False]
}
```

SVM (kernel):

```
svm_params = {
    "C": [0.1, 1.0, 10, 100, 500, 1000],
    "kernel": ["linear", "rbf", "poly"],
    "gamma": ["scale", "auto"]
}
```

SVM (linear):

```
lin_svm_params = {
    "C": [0.1, 1, 10, 100, 500, 1000, 5000, 10000],
    "loss": ["squared_hinge"]
}
```

Decision Tree:

```
dt_params = {
    'max_depth': [1, 5, 10, 20, 25, 30, 50],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 5],
    'criterion': ['gini', 'entropy']
}
```

Logistic Regression:

```
lr_params = {
    "C": [0.01, 0.1, 1, 10, 100, 200, 500, 1000],
    "solver": ["newton-cg", "lbfgs", "liblinear"]
}
```

- **Configuration of grid search with 10-fold cross-validation**

  As the dataset is moderately sized, `GridSearchCV` is configured to perform 10-fold cross validation with F1-score as the primary evaluation metric, aligning with the approach from Al-Kaabi et al. (2025) to improve model reliability and generalisability.

Figure 36

Configuration of `GridSearchCV` *for Decision Tree*

```
# Perform grid search to find for the best hyperparameter
dt_grid = GridSearchCV(
    DecisionTreeClassifier(random_state=1029),
    param_grid=dt_params,
    cv=10,
    scoring="f1",
    n_jobs=-1
)
```

Note. The same configuration is applied for other models.

- **Model retraining using optimal hyperparameters**

  Once obtaining the most optimal parameter combination, the models are retrained using these hyperparameters to re-evaluate its performance on training and testing datasets.

Figure 37

Model Retraining with Optimal Hyperparameter for Logistic Regression

```
# Apply the model on the training dataset and output performance (training)
lr_grid.fit(X_train_resampled, y_train_resampled)
print("Best Parameters:", lr_grid.best_params_)
print("Best CV F1-Score:", lr_grid.best_score_)

# Apply the best hyperparameters on the testing dataset and output results
best_lr = lr_grid.best_estimator_
best_lr_predict = best_lr.predict(X_test_final)
print("Optimized Test Accuracy:", accuracy_score(y_test, best_lr_predict))
print("Optimized Test Recall:", recall_score(y_test, best_lr_predict))
print("Optimized Test F1-Score:", f1_score(y_test, best_lr_predict))
print(classification_report(y_test, best_lr_predict))
```

Note. `lr_grid` refers to the `GridSearchCV` object created for the hyperparameter tuning of logistic regression. `X_train_resampled` represents the training features after preprocessing and SMOTE. `y_train_resampled` represents the target corresponding to `X_train_resampled` after SMOTE. The same steps are applied for other models.

The performance of each model before and after hyperparameter tuning is summarized in Table 6.

Table 6

Model Performance Comparison Before and After Hyperparameter Optimization

Model	Best Hyperparameters	Accuracy		Recall		F1-Score	
		Baseline	Optimized	Baseline	Optimized	Baseline	Optimized
Multinomial Naïve Bayes	• `alpha = 0.01` • `fit_prior = True`	0.9534	0.9623	0.9664	0.9262	0.8471	0.8679
Support Vector Machine (Kernel)	• `C = 1000` • `gamma = auto` • `kernel = rbf`	0.8287	0.9830	0.9060	0.8926	0.5857	0.9333
Support Vector Machine (Linear)	• `C = 5000` • `loss = squared_hinge`	0.9857	0.9857	0.9128	0.9195	0.9444	0.9448
Decision Tree	• `criterion = gini` • `max_depth = 25` • `min_samples_leaf = 1` • `min_samples_split = 2`	0.9587	0.9632	0.8792	0.8725	0.8506	0.8638
Logistic Regression	• `C = 500` • `solver = liblinear`	0.9722	0.9865	0.9262	0.9262	0.8990	0.9485

B.4 Model Evaluation and Discussion

B.4.1 Selection of Evaluation Metrics

Three evaluation metrics are selected to evaluate different model performance, namely accuracy, recall, and F1-score.

- **Accuracy**

  Accuracy is used to measure the ratio of correctly classified messages to the total number of test data. Despite commonly used to evaluate model performance, accuracy might provide misleading information if class distribution bias is not properly handled in imbalanced datasets (Vujovic, 2021).

- **Recall**

  Recall measures the true positive rate of the model, which is a key metric that reflects the model's ability to correctly identify *spam* messages, and is crucial in spam detection to ensure that no *spam* messages are missed during prediction.

- **F1-Score**

  F1-score is the primary metric considered during model evaluation as it balances precision and recall, making the predictive models more reliable especially when an imbalanced testing dataset is implemented (GeeksforGeeks, 2026).

B.4.2 Performance and Results Presentation

Table 7 shows the performance for the different models implemented in this project, where among the models, logistic regression has the highest accuracy, recall and F1-score.

Table 7

Comparison of Model Performance After Hyperparameter Tuning

Metrics \ Models	Naïve Bayes	SVM (Kernel)	SVM (Linear)	Decision Tree	Logistic Regression
Accuracy	0.9623	0.9830	0.9857	0.9632	0.9865
Recall	0.9262	0.8926	0.9195	0.8725	0.9262
F1-Score	0.8679	0.9333	0.9448	0.8638	0.9485

B.4.3 Critical Discussion on Model Performance

The key findings observed from Table 7 include:

- **Probabilistic and tree-based models show weaker predictive effectiveness**
  Overall, the F1-scores of the Naïve Bayes (0.8679) and decision tree (0.8638) models are lower compared with other models, indicating that these models have a higher tendency to misclassify *ham* and *spam* messages although their accuracies are approximately 96%. This reflects that both models may exhibit bias towards the majority class, since decision trees have the potential of overfitting when dealing with sparse data (Xing & Wang, 2025) while the multinomial Naïve Bayes model, with its low alpha value of 0.01 that indicates minimal Laplace smoothing, is also sensitive when handling rare features that impacts its robustness and generalisability. The inconsistency between these metrics indicates that the performances of both models are unstable when handling unseen data, thereby suggesting that these models are not suitable for spam message classification tasks.

- **Linear-based models demonstrate better and consistent performance**
  Unlike probability and tree-based models, the logistic regression and linear support vector machine (SVM) models exhibit strong performance with a relatively high accuracy and F1-score, reflecting the high capability of the models to correctly classify messages into ham or spam on the testing dataset. The TF-IDF vectorisation approach yields a high-dimensional, yet sparse feature set that linear models can handle effectively (Schaeffer et al., 2024) as logistic regression models implement regularization techniques that impose penalty towards large coefficients to ensure generalisability (Raschka, 2016), while linear SVM models also use a regularization parameter to balance between decision margin maximisation and model tolerance towards misclassification (Suárez et al., 2025). This allows linear-based models to gain a stable performance while achieving high accuracy and recall, making them ideal to be applied in spam classification task that prioritises on balancing between false positives and false negatives.

In summary, linear-based models offer a stronger and more consistent performance due to their ability to effectively handle sparse data. Among these models, logistic regression stands out to be the best model with the highest accuracy, recall and F1-score. Nonetheless, to improve the performance of these models, advanced techniques like ensemble models (see Appendix B) are also explored to capture the strengths of individual models and improve classification stability.

Conclusion

In summary, this report looks into the different NLP-related tasks, ranging from word tokenisation to sentence probabilities, before moving into building a text classification model for spam detection using the UCI SMS Spam Collection dataset. With careful hyperparameter tuning and model evaluation, the logistic regression algorithm stands out to be the best performing model with the highest accuracy, recall and F1-score, with enhancements like ensemble techniques (see Appendix B) explored. Nonetheless, as the project is limited to a medium-sized dataset with traditional machine learning models, further improvements may incorporate large-sized spam detection datasets with advanced deep learning techniques to improve prediction robustness and real-life applicability.

(3999 words)

References

Ahmed , N. (2023, September 11). *Tokenization techniques in NLP*. Retrieved from Comet ML: https://www.comet.com/site/blog/tokenization-techniques-in-nlp/

Airlangga, G. (2024). Optimizing SMS spam detection using machine learning: A comparative analysis of ensemble and traditional classifiers. *Journal of Computer Networks, Architecture and High Performance Computing, 6*(4), 1942-1951.

Ali, M. (2018). The human intelligence vs. artificial intelligence: Issues and challenges in computer assisted language learning. *International Journal of English Linguistics, 8*(5), 259-271.

Aljumily, R. (2024). Exploring the impact of stemming on text topic-based classification accuracy. *Journal of Linguistics, Culture and Communication, 2*(2), 204-224.

Al-Kaabi, H., Al-Ibraheemi, F., Jasim, A., Al-Rekabi, M., & Parchami, J. (2025). Fusion-based hybrid model for SMS spam detection integrating local, sequential, and contextual features. *Research Square*, 1-18.

Almeida, T., Hidalgo, J., & Yamakami, A. (2011). Contributions to the study of SMS spam filtering: New collection and results. *Proceedings of the 11th ACM Symposium on Document Engineering* (pp. 259-262). New York, USA: Association for Computing Machinery.

Bansode, R., & Pingle, S. (2025). SMS spam detection using machine learning: A hybrid SVM and Naive Bayes approach. *Journal of Emerging Technologies and Innovative Research, 12*(12), 437-440.

Bhuva, L. (2025, March 13). *Understanding decision trees and hyperparameter tuning in machine learning*. Retrieved from Medium: https://medium.com/@lomashbhuva/understanding-decision-trees-and-hyperparameter-tuning-in-machine-learning-c0a4467a1e69

Bird, S., Loper, E., & Klein, E. (2009). *Natural Language Processing with Python.* O'Reilly Media Inc.

Borade, N. (2026, January 5). *Naive Bayes algorithm: A complete guide with intuition, math, Python & tuning*. Retrieved from Medium: https://medium.com/@nikhilsborade0412/naive-bayes-algorithm-a-complete-guide-with-intuition-math-python-tuning-12c81a6430e9

Brownlee, J. (2020, August 28). *Tune hyperparameters for classification machine learning algorithms*. Retrieved from Machine Learning Mastery: https://machinelearningmastery.com/hyperparameters-for-classification-machine-learning-algorithms/

Ccoya, W., & Pinto, E. (2023). Comparative analysis of libraries for the sentimental analysis. *ArXiv, abs/2307.14311*, 1-5.

Chen, D. (2024). Challenges of natural language processing from a linguistic perspective. *International Journal of Education and Humanities, 13*(2), 217-219.

Dasgupta, A., & Mehr, S. (2024). Enhanced MNB method for spam e-mail/SMS text detection using TF-IDF vectorizer. *American Journal of Mathematical and Computer Modelling, 9*(1), 1-8.

Equbal, M. (2024, September 5). *Optimizing machine learning models: A deep dive into hyperparameter tuning techniques*. Retrieved from Medium: https://pub.towardsai.net/optimizing-machine-learning-models-a-deep-dive-into-hyperparameter-tuning-techniques-3cf90771508f

Flores, F., & Moreira, V. (2016). Assessing the impact of stemming accuracy on information retrieval – A multilingual perspective. *Information Processing & Management, 52*(5), 840-854.

Garg, M., Parveen, Gupta, M., & Ojasvi. (2022). Email spam detection using logistic regression. *Journal of Pharmaceutical Negative Results, 13*(10), 2111-2118.

GeeksforGeeks. (2025, December 17). *Introduction to stemming*. Retrieved from GeeksforGeeks: https://www.geeksforgeeks.org/machine-learning/introduction-to-stemming/

GeeksforGeeks. (2026, March 30). *Evaluation metrics in machine learning*. Retrieved from GeeksforGeeks: https://www.geeksforgeeks.org/machine-learning/metrics-for-machine-learning-model/

Gichere , F. (2023, October 4). *Sentiment analysis of app reviews: A comparison of BERT, spaCy, TextBlob, and NLTK*. Retrieved from Medium: https://becominghuman.ai/sentiment-analysis-of-app-reviews-a-comparison-of-bert-spacy-textblob-and-nltk-9016054d54dc

Guskov, S. (2023, March 3). *Comparison of NLP tokenizers: Choosing the best option for your needs*. Retrieved from Medium: https://medium.com/@gusevski.dev/comparison-of-nlp-tokenizers-choosing-the-best-option-for-your-needs-f7aa4d18caa6

Hankare, O. (2024, March 8). *Exploring n-grams: The building blocks of natural language understanding*. Retrieved from Medium: https://ompramod.medium.com/exploring-n-grams-the-building-blocks-of-natural-language-understanding-b9b90dffe0a2

Hickman, L., Thapa, S., Tay, L., Cao, M., & Srinivasan, P. (2022). Text preprocessing for text mining in organizational research: Review and recommendations. *Organizational Research Methods, 25*(1), 114-146.

Imdadullah, M., Aslam, M., & Altaf, S. (2016). mctest: An R package for detection of collinearity among regressors. *The R Journal, 8*(2), 495-505.

Jivani, A. (2011). A comparative study of stemming algorithms. *Int. J. Comp. Tech. Appl., 2*(6), 1930-1938.

Johari, M., Chiew, K., Hosen, A., Yong, K., Khan, A., Abbasi, I., & Grzonka, D. (2025). Key insights into recommended SMS spam detection datasets. *Scientific Reports, 15*, 8162.

Jurafsky, D., & Martin, J. (2026). *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models.*

Kaur, J., & Buttar, P. (2018). A systematic review on stopword removal algorithms. *International Journal on Future Revolution in Computer Science & Communication Engineering, 4*(4), 207-210.

Miller, G. (2025). Evaluating SMS spam classification: Human judgement vs machine learning models. 1-11.

Min, J., Lee, M., Lee, W., & Lee, Y. (2025). Punctuation restoration improves structure understanding without supervision. *Proceedings of the 10th Workshop on Representation Learning for NLP* (pp. 120-130). Association for Computational Linguistics.

Murel, J., & Kavlakoglu, E. (2023, November 24). *What is stemming?* Retrieved from IBM: https://www.ibm.com/think/topics/stemming

Nafis, N., & Awang, S. (2019). The impact of pre-processing and feature selection on text classification. *Advances in Electronics Engineering. 619*, pp. 269-280. Singapore: Springer.

Navlani, A. (2022, April 9). *Explore Python Gensim library for NLP*. Retrieved from Medium: https://avinashnavlani.medium.com/explore-python-gensim-library-for-nlp-a1adcea2bb8c

Neumann, M., King, D., Beltagy, I., & Ammar, W. (2019). ScispaCy: Fast and robust models for biomedical natural language processing. *Proceedings of the 18th BioNLP Workshop and Shared Task* (pp. 319-327). Florence, Italy: Association for Computational Linguistics.

Nguyen, Q. (2022). *Naive Bayes and Hyperparameter Optimization*. Retrieved from BAIT 509: Business Applications of Machine Learning: https://bait509-ubc.github.io/BAIT509/lectures/lecture6.html

Omar, M., Choi, S., Nyang, D., & Mohaisen, D. (2022). Robust natural language processing: Recent advances, challenges, and future directions. *IEEE Access, 10*, 86038-86056.

Patil, S. (2025). Enhancing email fraud detection using SVM: A comparative analysis with other machine learning models. *International Journal for Multidisciplinary Research, 7*(2), 1-8.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., . . . Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825-2830.

Pham, B. (2020). Parts of speech tagging: Rule-based. 1-6.

PythonPI. (2023, October 30). *sacremoses 0.1.1*. Retrieved from PythonPI: https://pypi.org/project/sacremoses/

Raschka, S. (2016, June 24). *Regularization in logistic regression: Better fit and better generalization?* Retrieved from KDnuggets: https://www.kdnuggets.com/2016/06/regularization-logistic-regression.html

Sarica, S., & Luo, J. (2021). Stopwords in technical language processing. *PLoS ONE, 16*(8), e0254937.

Schaeffer, J., Lenz, E., Chueh, W., Bazant, M., Findeisen, R., & Braatz, R. (2024). Interpretation of high-dimensional linear regression: Effects of nullspace and regularization demonstrated on battery data. *Computers & Chemical Engineering, 180*, 108471.

Shaikh, A., Shaikh, M., & Srivaramangai, R. (2025). Comparative analysis of machine learning models for smishing message detection. *International Journal of Science and Research, 14*(12), 1984-1990.

Suárez, C., Castro, M., Leon, M., Martin-Barreiro, C., & Liut, M. (2025). Improving SVM performance through data reduction and misclassification analysis with linear programming. *Complex & intelligent systems, 11*(8), 356.

Tusher, E., Ismail, M., & Raffei, A. (2025). Email spam classification based on deep learning methods: A review. *Iraqi Journal for Computer Science and Mathematics, 6*(1), 24-36.

Vujovic, Z. (2021). Classification model evaluation metrics. *International Journal of Advanced Computer Science and Applications, 12*(6), 599-606.

Xing, Q., & Wang, Y. (2025). Research on text classification methods based on decision trees: A case study on the recognition of the entity category 'Position'. *Academic Journal of Natural Science, 2*(2), 10-15.

Yadav, D., & Kaur, E. (2023). Machine learning models for email spam detection: A review. *2023 Second International Conference On Smart Technologies For Smart Nation (SmartTechCon)*, (pp. 1109-1114). Singapore.

Appendix A

Workload Matrix

Task		Contribution Percentage			
		Chan Chun Ming (TP068983)	**Eu Jun Hong (TP068580)**	**Lim Beng Rhui (TP068495)**	**Ng Xuan Jack (TP067678)**
Introduction		-	-	100%	-
Part A	Tokenisation and Text Filtering	-	-	100%	-
	Word Stemming	100%	-	-	-
	POS Tagging and Parse Trees Analysis	-	100%	-	-
	Sentence Probability Analysis	-	-	-	100%
	Alternative Tokenisation Approach	25%	25%	25%	25%
Part B	Dataset Selection and Exploratory Data Analysis	-	30%	40%	30%
	Supervised Text Classification	-	-	100%	-
	Hyperparameter Selection and Optimisation	-	-	100%	-
	Model Evaluation and Discussion	100%	-	-	-
Conclusion		-	-	100%	-
Overall Contribution		**27.5%**	**20%**	**32.5%**	**20%**

Appendix B

Ensemble Model

An ensemble model that combines all the four predictive models is also developed to improve the overall robustness of model, as the ensemble model is able to pick up each model's strengths while reducing the impacts caused by their respective weaknesses. As shown, the majority voting strategy is applied to the ensemble model, where each model contributes a vote during prediction, and the class with the highest vote is selected as the final output. The ensemble model is able to achieve an overall accuracy of 99.01% with a recall of 0.9262 and a F1-score of 0.9617, slightly outperforming the individual models in terms of accuracy and F1-score.

```python
# Recreate models based on the best hyperparameter obtained
best_nb_model = MultinomialNB(
    alpha=nb_grid.best_params_["alpha"]
)
best_svm_model = SVC(
    C=svm_grid.best_params_["C"],
    kernel=svm_grid.best_params_["kernel"],
    gamma=svm_grid.best_params_["gamma"]
)
best_lin_svm_model = LinearSVC(
    C=lin_svm_grid.best_params_["C"],
    loss=lin_svm_grid.best_params_["loss"]
)
best_dt_model = DecisionTreeClassifier(
    max_depth=dt_grid.best_params_["max_depth"],
    min_samples_split=dt_grid.best_params_["min_samples_split"],
    criterion=dt_grid.best_params_["criterion"],
    random_state=1029
)
best_lr_model = LogisticRegression(
    C=lr_grid.best_params_["C"],
    solver=lr_grid.best_params_["solver"],
    max_iter=1000
)

# Create an ensemble model using majority voting
ensemble_model = VotingClassifier(
    estimators=[
        ("nb", best_nb_model),
        ("svm", best_svm_model),
        ("dt", best_dt_model),
        ("lr", best_lr_model),
        ("lin_svm", best_lin_svm_model),
    ],
    voting="hard"
)

# Train the ensemble model with training dataset and perform prediction on testing dataset
ensemble_model.fit(X_train_resampled, y_train_resampled)
ensemble_predict = ensemble_model.predict(X_test_final)

# Output the accuracy, recall and F1-score of the model
print("Ensemble Model Accuracy:", accuracy_score(y_test, ensemble_predict))
print("Ensemble Model Recall:", recall_score(y_test, ensemble_predict))
print("Ensemble Model F1-Score:", f1_score(y_test, ensemble_predict))
print("\n", classification_report(y_test, ensemble_predict))
```

✓ [69] 3s 536ms

```
Ensemble Model Accuracy: 0.9901345291479821
Ensemble Model Recall: 0.9261744966442953
Ensemble Model F1-Score: 0.9616724738675958

              precision    recall  f1-score   support

           0       0.99      1.00      0.99       966
           1       1.00      0.93      0.96       149

    accuracy                           0.99      1115
   macro avg       0.99      0.96      0.98      1115
weighted avg       0.99      0.99      0.99      1115
```